

[illegible]

WEB SERVICES

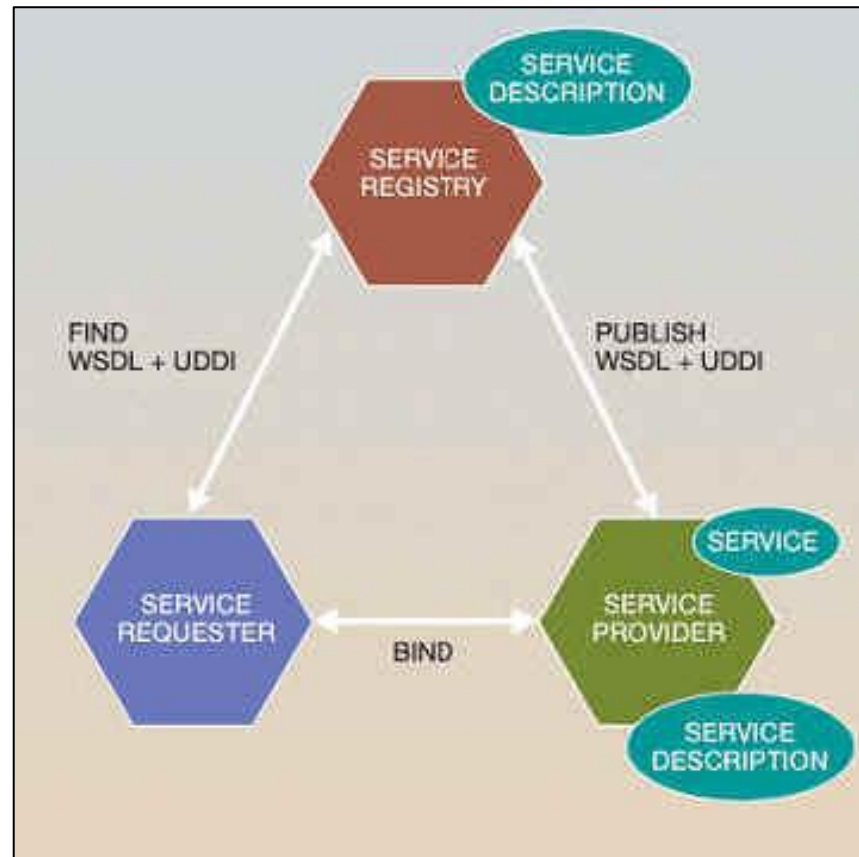
REST & SOAP INTRODUZIONE

WEB SERVICES

INTRODUZIONE ALLE ARCHITETTURE A WS

Architetture e Tecnologie basilari dei Web Services

- L'architettura che sta alla base dei Web services è ben definita e sempre la stessa, indipendentemente dai Web services specifici dei vari fornitori utilizzati.
- L'architettura descrive le interazioni tra tre componenti principali:
 - Service Provider
 - Service Requester
 - Service Registries



Architetture e Tecnologie basilari dei Web Services

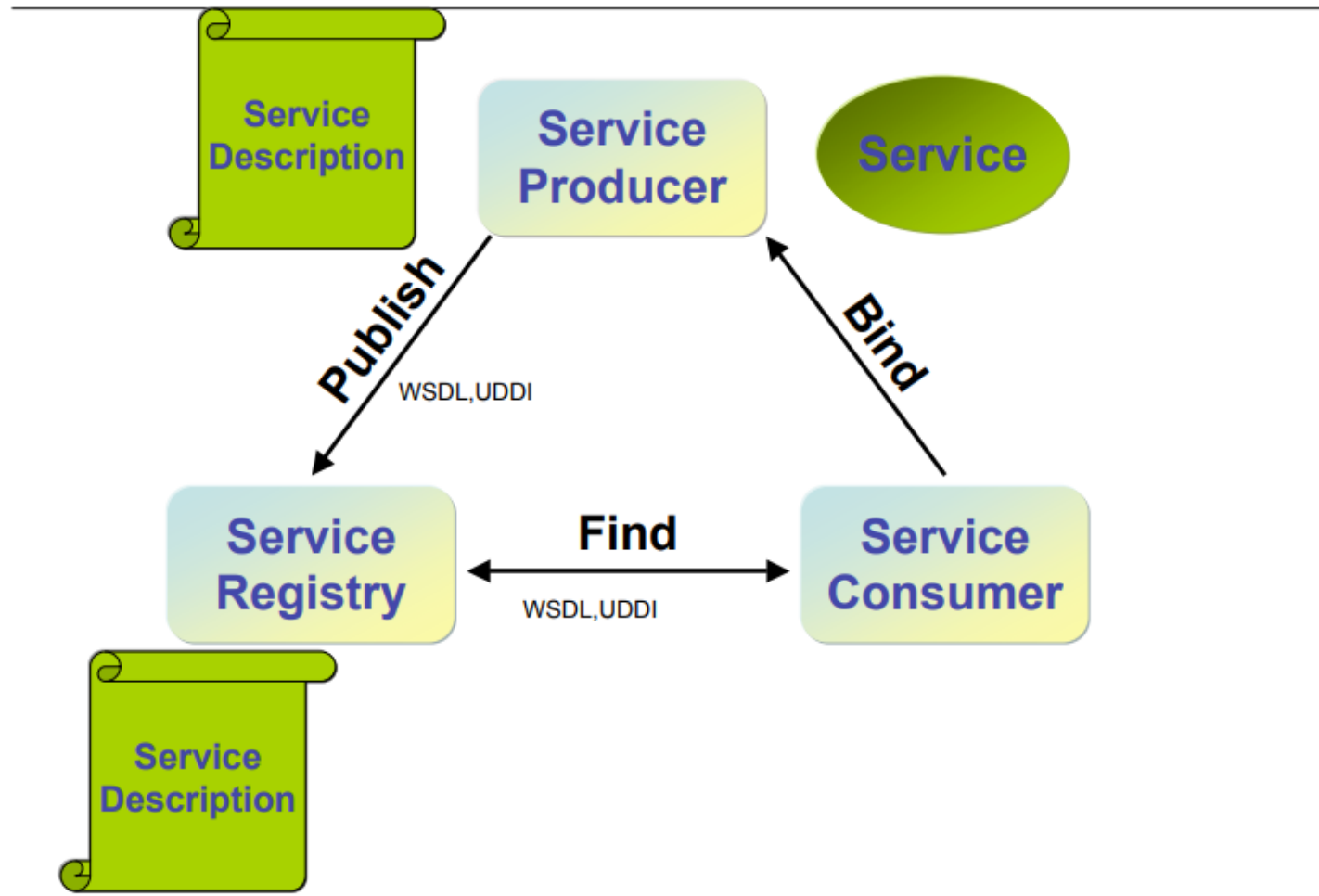
- **Service Provider:** Sviluppa e definisce i Web Services e li pubblica poi all'interno dei Web Service Registries, oppure li rende direttamente disponibili ai Web Service Requester. E' una piattaforma che fornisce l'accesso al servizio. Per esporre un'applicazione come un servizio, il provider deve fornire un accesso basato su un protocollo standard e una descrizione standard del servizio (meta data).
- **Service Requester o user:** Effettua un'operazione di ricerca per localizzare i servizi desiderati resi disponibili dai Web Service Provider e poi richiedono questi stessi servizi ai Web Service Registries oppure direttamente al luogo della loro pubblicazione. Una volta localizzato, il Web Service Requester si connette al suddetto servizio. In generale, è un'applicazione che invoca o avvia un'interazione con un servizio. Il service requestor e il service provider devono condividere le stesse modalità di accesso e interazione col servizio.
- **Service Registries:** Si comportano come directory centralizzate, e sono depositari dei Web Services definiti e pubblicati dai Web Service Provider. E' un registro presso il quale i service provider possono pubblicare il servizio e i service requestor possono trovare i servizi desiderati. Il service registry deve fornire una tassonomia consistente dei Web Services per facilitarne la scoperta, e una descrizione dettagliata dell'azienda che fornisce il servizio e del servizio stesso.

Architetture e Tecnologie basilari dei Web Services

- Le operazioni che consentono l'interazione tra le tre componenti del modello sono:
 - Publish: Il service provider invia la descrizione del servizio, che ospita, al service registry. Questo classifica il servizio e lo pubblica in maniera tale che un service requestor possa trovarlo.
 - Find: Il service requestor interroga il service registry per ottenere la descrizione del servizio richiesto.
 - Bind: Il service requestor utilizza i dettagli presente nella descrizione del servizio per contattare il service provider mediante il quale interagisce con l'implementazione del servizio.

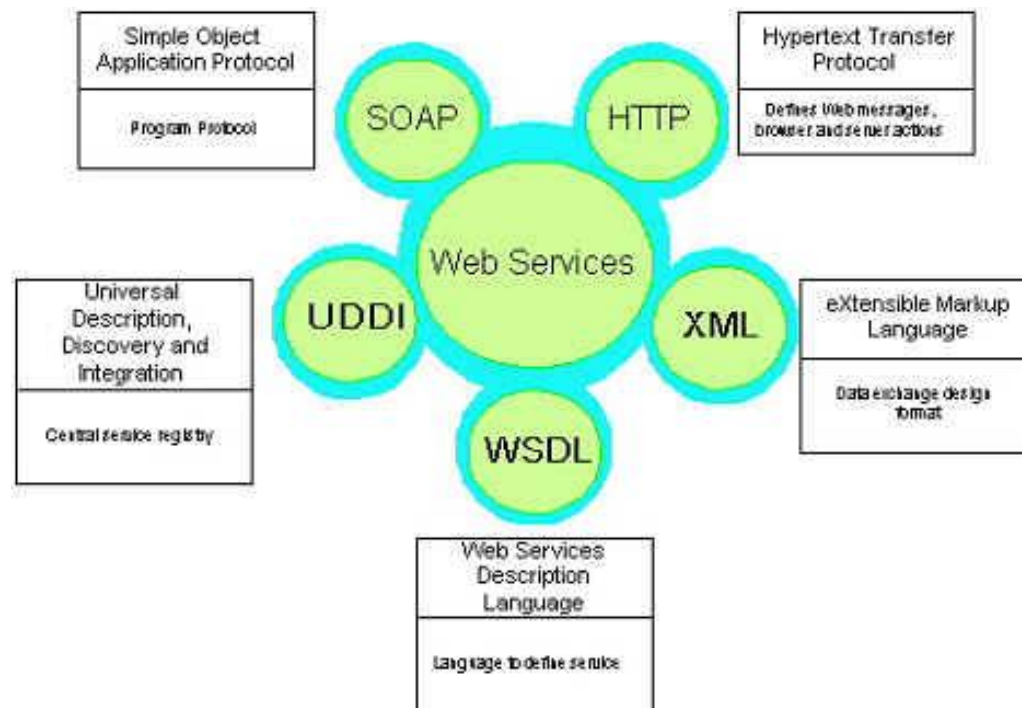
Architetture e Tecnologie basilari dei Web Services

The Web Services Model -



Architetture e Tecnologie basilari dei Web Services

- Il fatto che l'industria si sta muovendo verso standard di programmazione (Java) e di scambio dei dati (XML) comuni, ha favorito l'ascesa dei Web Services. E' assolutamente essenziale che gli standard nell'area dei Web Services siano ovunque gli stessi; senza l'accordo dell'industria di tutto il mondo sugli standard della tecnologia sottostante ai Web Services, la promessa della neutralità delle piattaforme e della compatibilità universale di sistemi eterogenei non può semplicemente essere mantenuta.
- Ulteriori standard per chiamate di procedure remote (SOAP) e chiamate di directory (UDDI) e di servizi di descrizione (WSDL) completano la gamma delle tecnologie dei Web Services.



Cosa possono fare i Web Services per noi

- I Web Services possono collegarci ai nostri impiegati, clienti, partner e fornitori offrendo un'integrazione totale, un'interoperabilità indipendente e una sicurezza intrinseca.
- Grazie ai Web Services possiamo facilmente sviluppare e pubblicare nuove applicazioni che saranno compatibili con tutte le piattaforme e server, e così flessibili da poter essere riutilizzati.
- Naturalmente nel momento in cui si entra nel mondo dei Web services, si sarà anche abilitati a trarre vantaggio dai servizi pubblicati da altre organizzazioni, senza doversi preoccupare se i servizi a cui si sta attingendo siano compatibili con il proprio sistema, perchè i servizi sono stati sviluppati per gli open standard Web Services che già si supportano.
- Le possibilità di ciò che è possibile creare con i Web Services sono virtualmente illimitate. Un Web Service può avere qualsiasi capacità definibile con un applicazione portatile, da una semplice richiesta e operazione di calcolo, ad un complesso procedimento business.

Cosa possono fare i Web Services per noi

- Per illustrare la varietà delle possibilità dei Web Services, qui sono riportati alcuni esempi di funzioni che possono essere definite come Web Services:
 - modulo di richiesta permessi
 - verifica dello stato di avanzamento ordini
 - rilevamento della rotazione dei prodotti a magazzino
 - storico dei conti dei clienti
 - aggiornamento degli indirizzi dei clienti
- Quindi, in breve, qualsiasi funzione semplice o complessa, definibile con una componente discreta e portatile può essere creata come Web Service.
- Una volta definito e pubblicato può essere trovato e utilizzato da chiunque ne necessiti, senza alcun rischio per il sistema residente alla base.

Integrare i sistemi legacy con i Web Services

- I Web Services sono particolarmente indicati quando si vuole integrare i sistemi legacy con le nuove applicazioni e-business, perché è possibile valorizzare questi importanti asset esistenti, senza interrompere o mettere a rischio il sistema legacy mission critical. L'approccio è possibile in due modi: accesso alla transazione diretta, oppure accesso screen-based.
- Quando la presentazione e la business logic in un'applicazione legacy sono chiaramente separate, i Web Services offrono un ottimo modo di accedere direttamente alle transazioni, per poi potenziare e incapsulare le logiche applicative già esistenti all'interno delle nuove applicazioni utilizzabili a più ampio spettro.
- L'accesso screen-based rappresenta un metodo semplice e veloce di integrazione di Web Services e sistemi legacy, soprattutto quando la presentazione e la business logic delle applicazioni sono strettamente interconnesse tra di loro, cosa piuttosto frequente.
- I fornitori offrono dei tool per il potenziamento rapido e scorrevole sia per le transazioni, sia per le screen-based, per poterle poi sviluppare all'interno dei Web Services.
- Questi tool permettono a sviluppatori con esperienza di applicazioni legacy di fornire velocemente e facilmente Web Services legacy-integrated pronti per essere utilizzati da sviluppatori Web meno abituati ai sistemi host legacy.

Sviluppo rapido di nuove applicazioni

- Si può velocemente sviluppare Web services con i tool di sviluppo delle applicazioni già conosciuti.
- Le infrastrutture delle applicazioni Web più usate, comprese Java 2 Enterprise Edition (J2EE), Microsoft.Net e IBM WebSphere, offrono ampio supporto per gli standard dei Web Services.
- E in aggiunta, i tool di sviluppo Web messi a disposizione dai fornitori rendono lo sviluppo dei Web Services ancora più facile e automatico.
- Spesso è sufficiente definire le interfacce e le funzioni desiderate, e il software genera automaticamente il codice necessario.
- Estrahendo regole di business complesse e distribuendole in una gerarchia di servizi decentralizzati, si possono creare nuove applicazioni e-business, risparmiando tempo e impegno degli sviluppatori.

Flessibilità e riutilizzabilità

- Un Web Service sviluppato con uno scopo preciso può essere facilmente riutilizzato in altre situazioni, proprio in virtù della sua conformità agli standard dei Web Services.
- Un unico Web Service può essere usato in numerose situazioni, ovunque all'interno di una stessa azienda, così come può essere combinato con altri Web Services.
- Finché a livello dello sviluppo è supportata la tecnologia dei Web Services, è garantita la flessibilità di costruire delle librerie di oggetti di applicazioni, riutilizzabili in numerose applicazioni e-business personalizzate, indipendentemente da questioni di compatibilità delle piattaforme.

La sicurezza intrinseca dei Web Services

- I Web Services forniscono ai proprietari di vari sistemi enterprise il controllo su quali dati e funzioni vogliono rendere accessibili ad altre applicazioni o a terze parti.
- Sono loro a scegliere e ad accettare di supportare solamente quelle funzioni che desiderano esporre all'utilizzo, mantenendo sempre la capacità di gestire la sicurezza e l'accesso al loro sistema.
- Inoltre i Web Services fungono intrinsecamente come applicazioni firewall per i sistemi enterprise.
- Con i Web Services si può esercitare un controllo granulare su quali logiche e dati esattamente si vogliono esporre all'accesso di terze parti.
- Queste parti non potranno accedere ai sistemi sottostanti, in quanto questi ultimi non saranno mai esposti in prima linea.

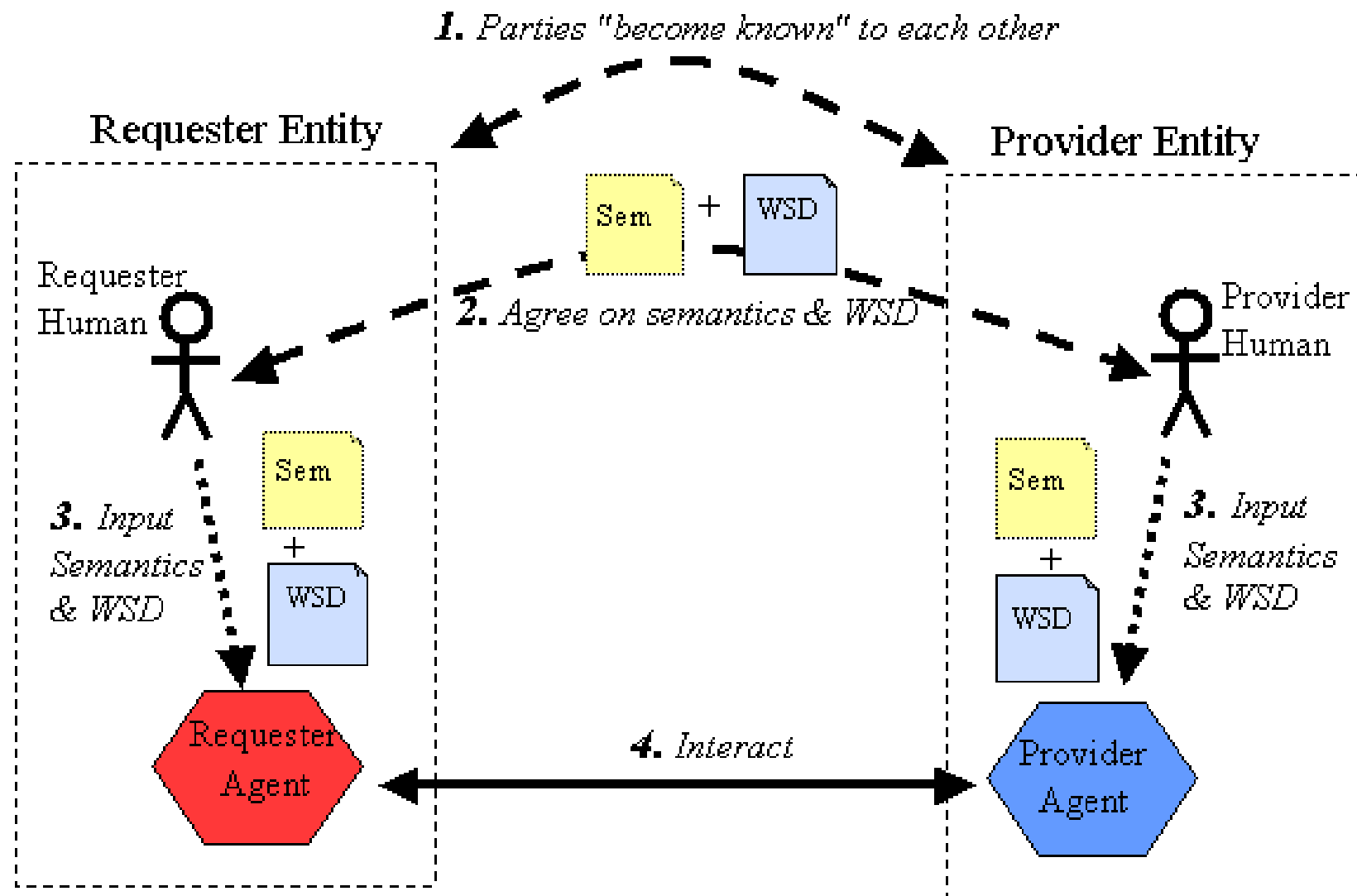
W3C Services Architecture

- Il World Wide Web Consortium, anche conosciuto come W3C, è un'organizzazione non governativa internazionale che ha come scopo quello di sviluppare tutte le potenzialità del World Wide Web.
- Al fine di riuscire nel proprio intento, la principale attività svolta dal W3C consiste nello stabilire standard tecnici per il World Wide Web inerenti sia i linguaggi di markup che i protocolli di comunicazione.
- Il World Wide Web Consortium è stato fondato nell'ottobre del 1994 presso il MIT (Massachusetts Institute of Technology) dal "padre" del Web, Tim Berners-Lee, in collaborazione con il CERN (il laboratorio dal quale Berners-Lee proveniva).
- Per comodità, le attività del W3C sono state divise in 4 aree di lavoro (in gergo chiamate domini):
 - **Architecture Domain**: ha il compito di gestire la tecnologia che è alla base del web.
 - **Interaction Domain**: cerca di semplificare l'interazione uomo-informazioni ed il modo di connettersi al web.
 - **Technology and Society Domain**: ha il ruolo di adattare l'infrastruttura tecnologica agli interessi sociali, legali e pubblici.
 - **Web Accessibility Initiative** (WAI): il suo lavoro è garantire che chiunque possa sfruttare appieno le potenzialità del web. Il suo lavoro si articola in 5 aree tematiche: tecnologia, linee guida, strumenti, educazione ed aiuto ai bisognosi, ricerca e sviluppo.
- In più esiste la **Quality Assurance** che formalizza i progressi fatti dai domini in documenti (chiamati Recommendations) che sono la principale fonte d'informazione per chiunque voglia creare nel web, e la Patent Policy che si assicura che questi progressi non violino alcun diritto d'autore o brevetto.

W3C Services Architecture

- Per quanto concerne i Servizi Web, il W3C ne definisce l'architettura ovvero identifica i componenti funzionali e definisce le relazioni tra questi componenti per ottenere le proprietà desiderate dell'architettura generale.
- Il W3C pone come scopo di un'architettura di un servizio web nel suo documento ufficiale (denominato WSA):
 - I servizi Web forniscono uno strumento standard per l'interoperabilità tra diverse applicazioni software, eseguendo su una varietà di piattaforme e / o framework. Il WSA ha lo scopo di fornire una definizione comune di un servizio Web e definirne la posizione all'interno di un più ampio framework di servizi Web per guidare la comunità. WSA fornisce un modello concettuale e un contesto per comprendere i servizi Web e le relazioni tra i componenti di questo modello.
 - L'architettura non tenta di specificare come vengono implementati i servizi Web e non impone alcuna limitazione su come i servizi Web potrebbero essere combinati. La WSA descrive sia le caratteristiche minime comuni a tutti i servizi Web, sia una serie di caratteristiche che sono necessarie per molti, ma non per tutti, i servizi Web.
 - L'architettura dei servizi Web è un'architettura di interoperabilità : identifica quegli elementi globali della rete di servizi Web globali necessari per garantire l'interoperabilità tra i servizi Web.
- Per la W3C un servizio Web è un sistema software progettato per supportare l'interazione interoperabile macchina-macchina su una rete. Ha un'interfaccia descritta in un formato elaborabile dalla macchina (in particolare WSDL). Altri sistemi interagiscono con il servizio Web in un modo prescritto dalla sua descrizione utilizzando i messaggi SOAP, in genere trasmessi tramite HTTP con una serializzazione XML in combinazione con altri standard relativi al Web.

Il processo generale di coinvolgimento di un servizio We

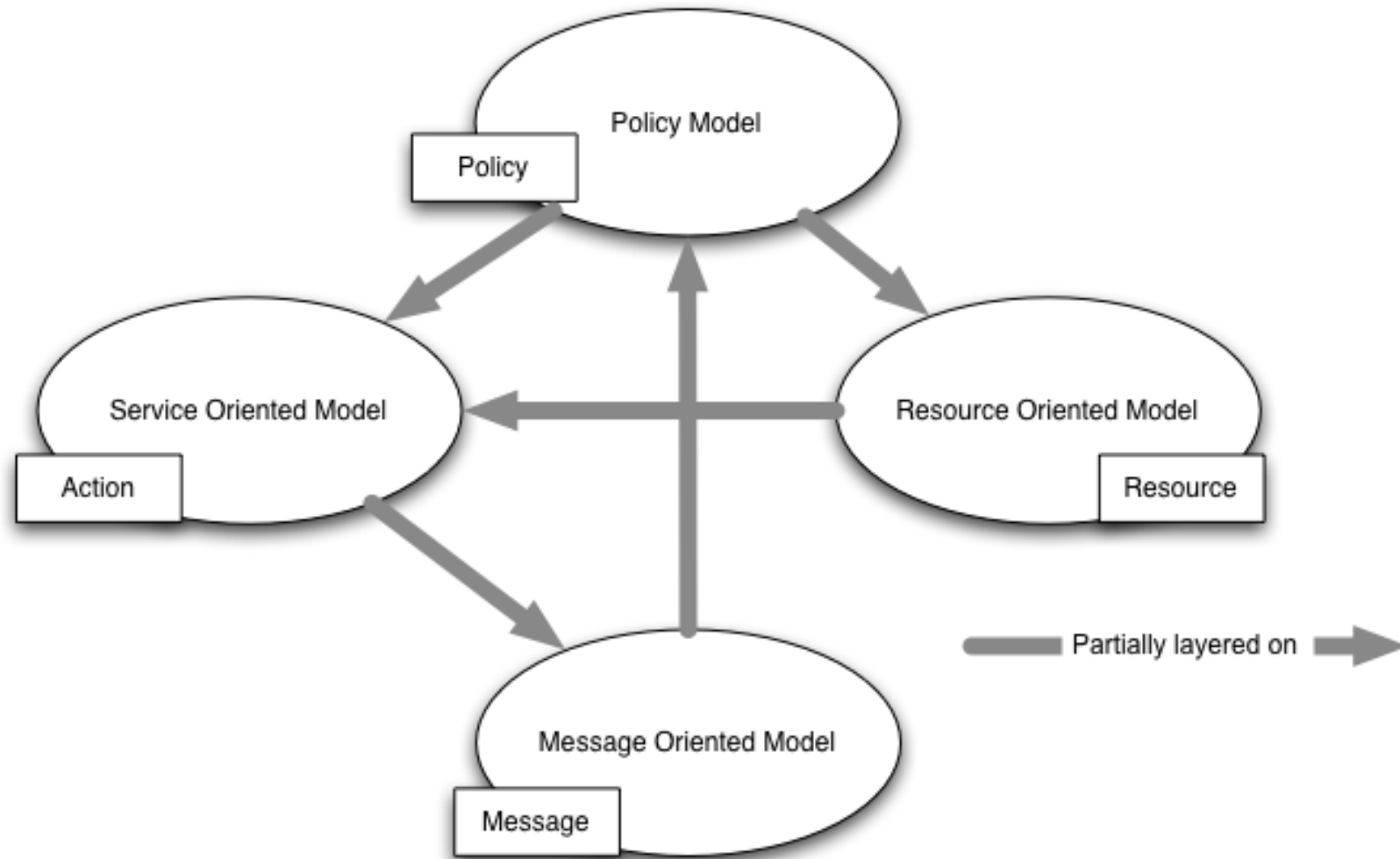


WEB SERVICES

W3C – Architettura dei Servizi

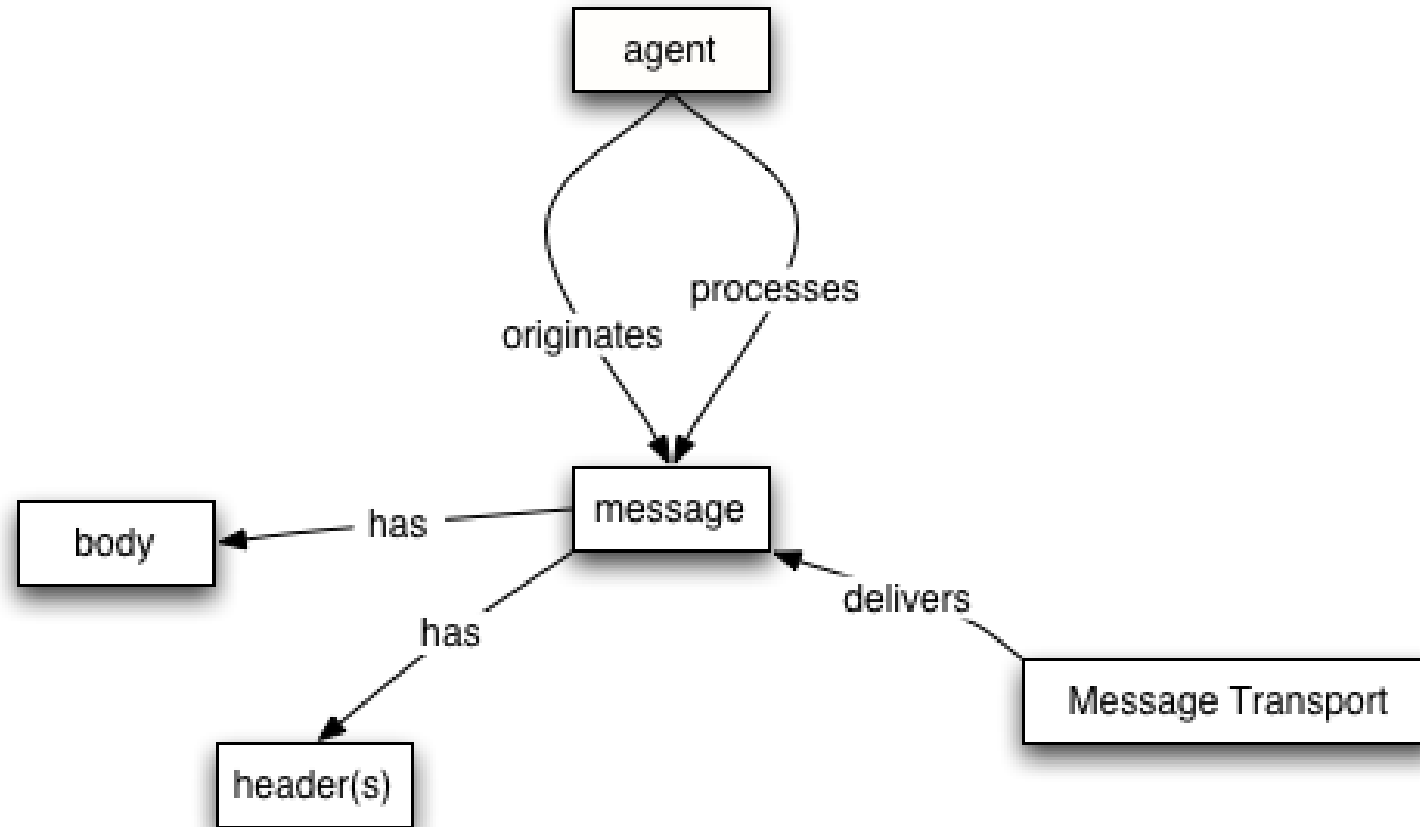
Meta modello dell'architettura

- La W3C ha disegnato l'architettura dei W3C poggiando su quattro tipi di modello di riferimento:



Modello orientato ai messaggi (MOM)

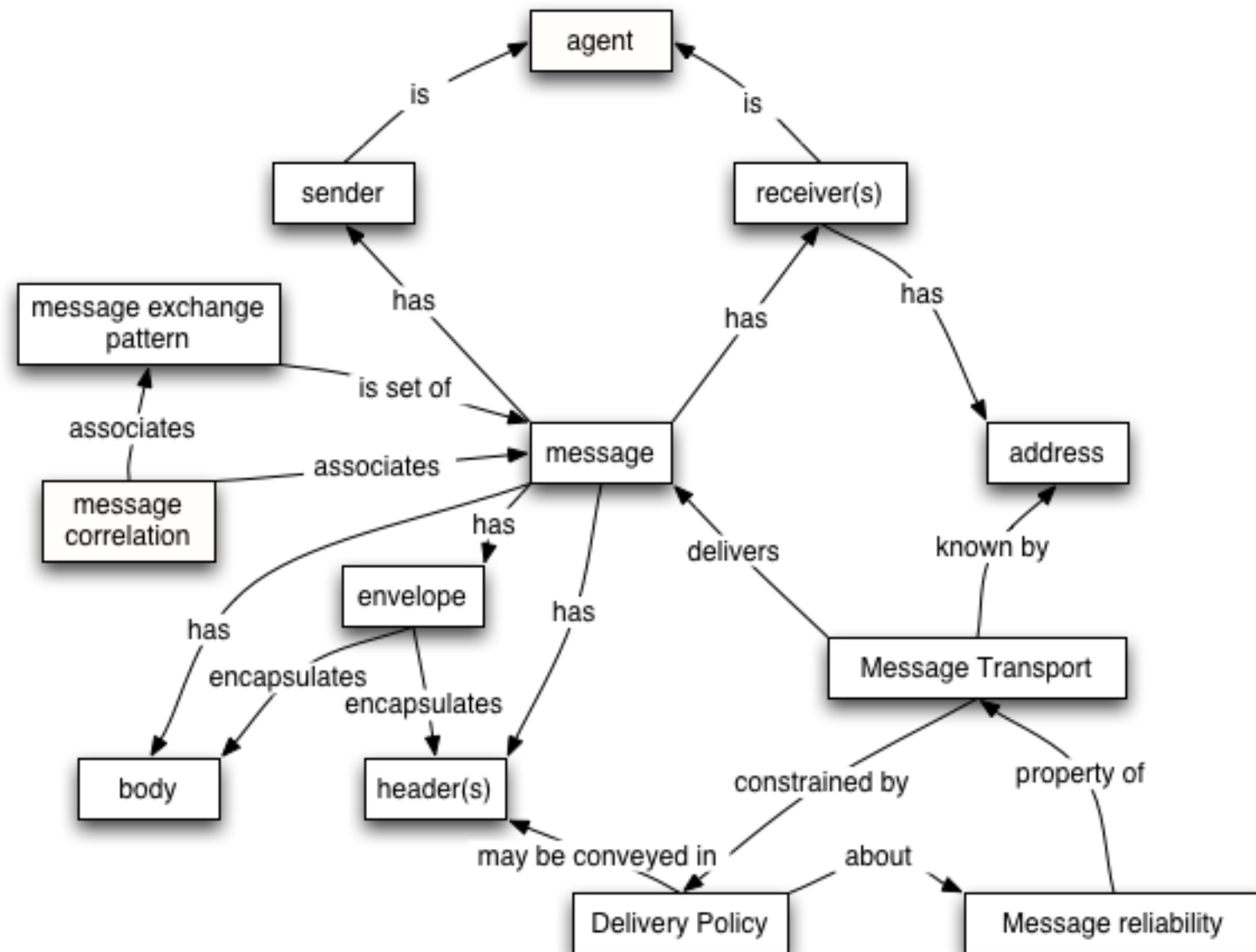
- Il modello orientato ai messaggi si concentra sui messaggi, sulla struttura dei messaggi, sul trasporto dei messaggi e così via, senza particolare riferimento alle ragioni dei messaggi o alla loro importanza.



Modello orientato ai messaggi (MOM) – Concetti Base

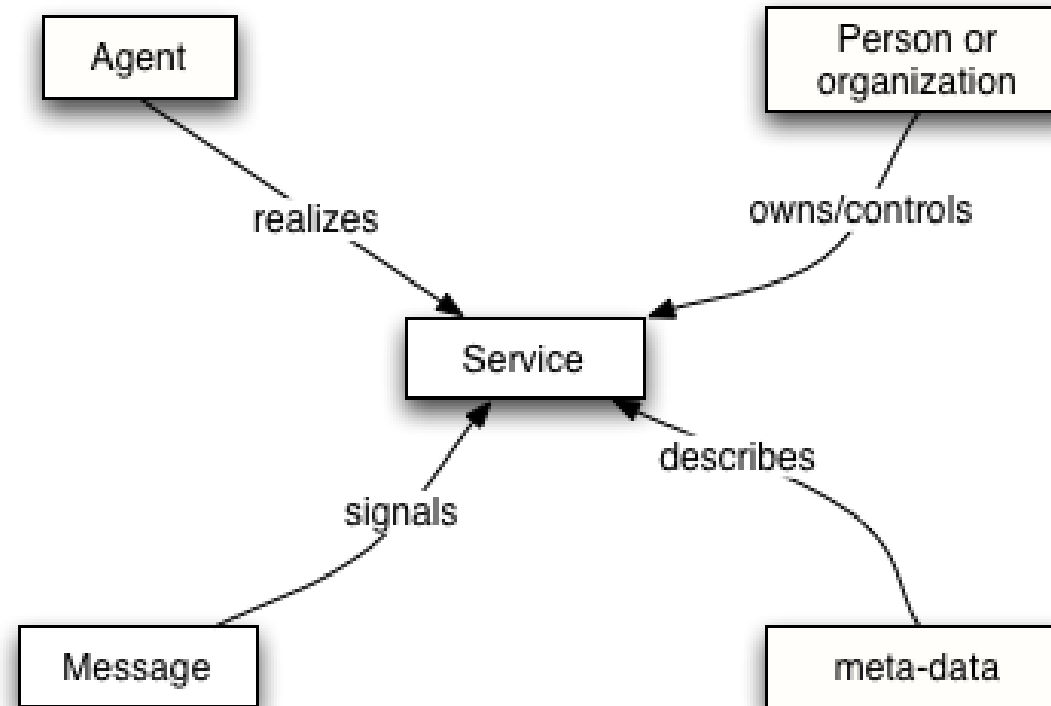
1. Address
2. Message
3. Body Message
4. Message Envelope (Consegna Busta)
5. Message Exchange Pattern (MEP)
6. Message Header
7. Message Receiver
8. Message Reliability (Affidabilità del messaggio)
9. Message Sender
10. Message Sequence
11. Message Transport

Modello orientato ai messaggi (MOM) – Dettaglio



Modello orientato ai Servizi (SOM)

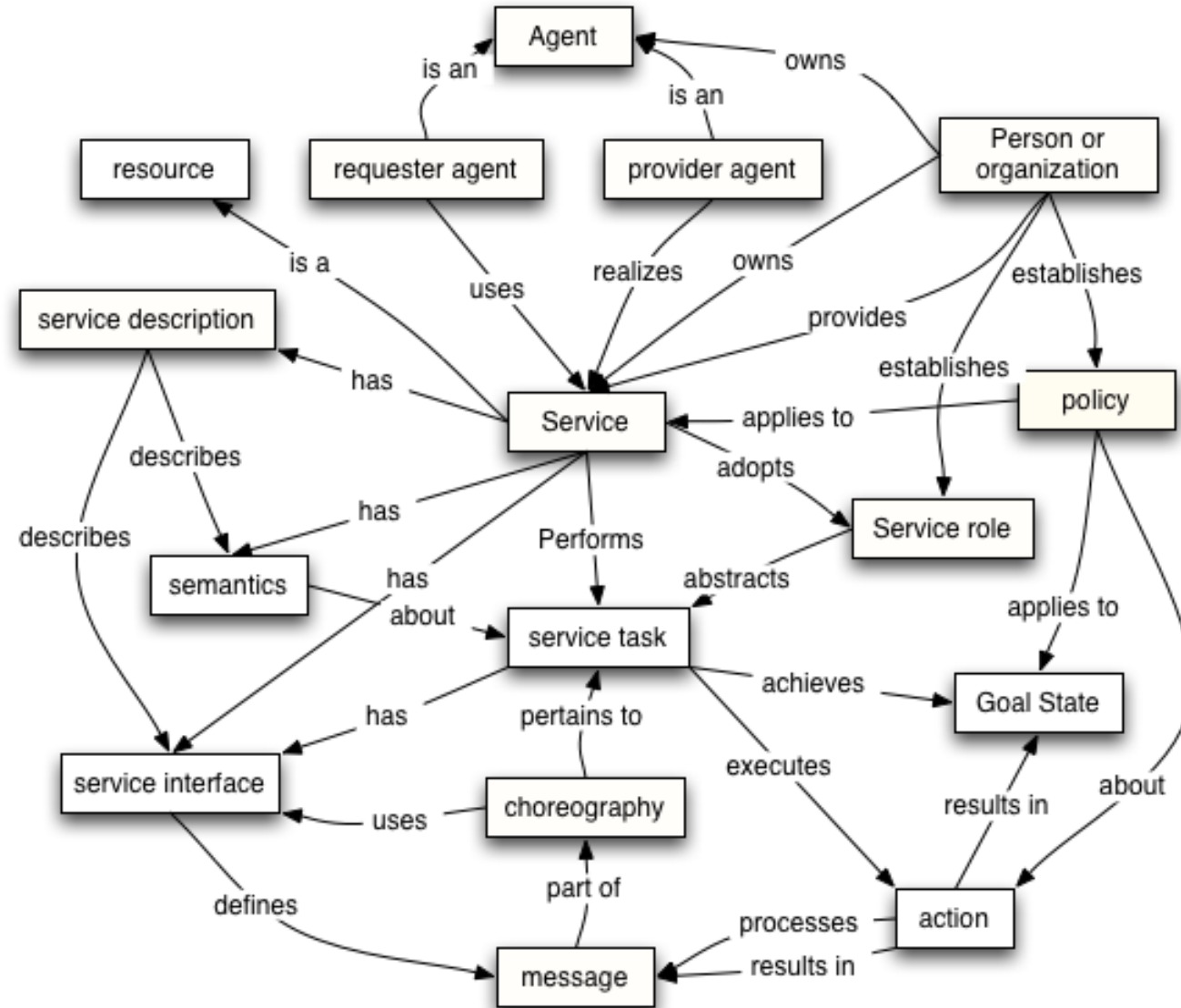
- Il modello orientato ai servizi si concentra su aspetti di servizio, azione e così via. Mentre chiaramente, in qualsiasi sistema distribuito, i servizi non possono essere realizzati in modo adeguato senza alcun mezzo di messaggistica, il contrario non è vero: i messaggi non devono necessariamente riguardare i servizi.



Modello orientato ai Servizi (SOM) – Concetti Base

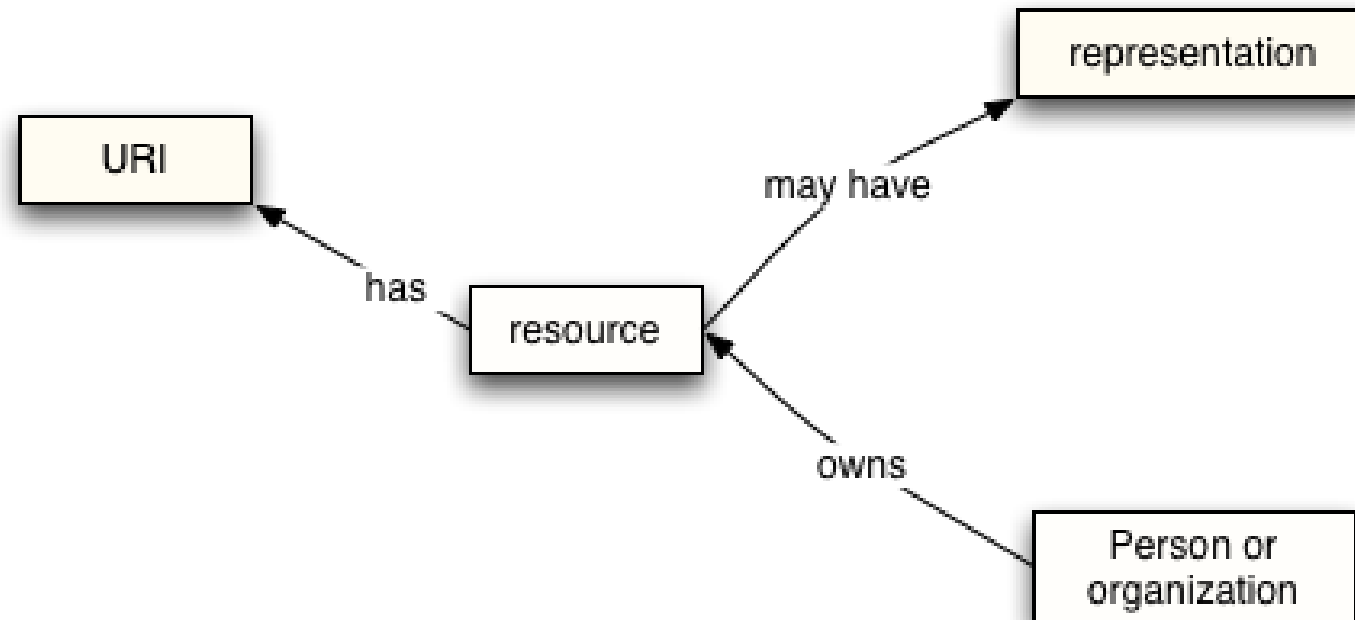
1. Action
2. Agent
3. Choreography
4. Capability
5. Goal State
6. Provider Agent
7. Provider Entity
8. Requester Agent
9. Requester Entity
10. Service
11. Service Description
12. Service Interface
13. Service Intermediary
14. Service Role
15. Service Semantics
16. Service Task

Modello orientato ai Servizi (SOM) – Dettaglio



Modello orientato alla risorsa (ROM)

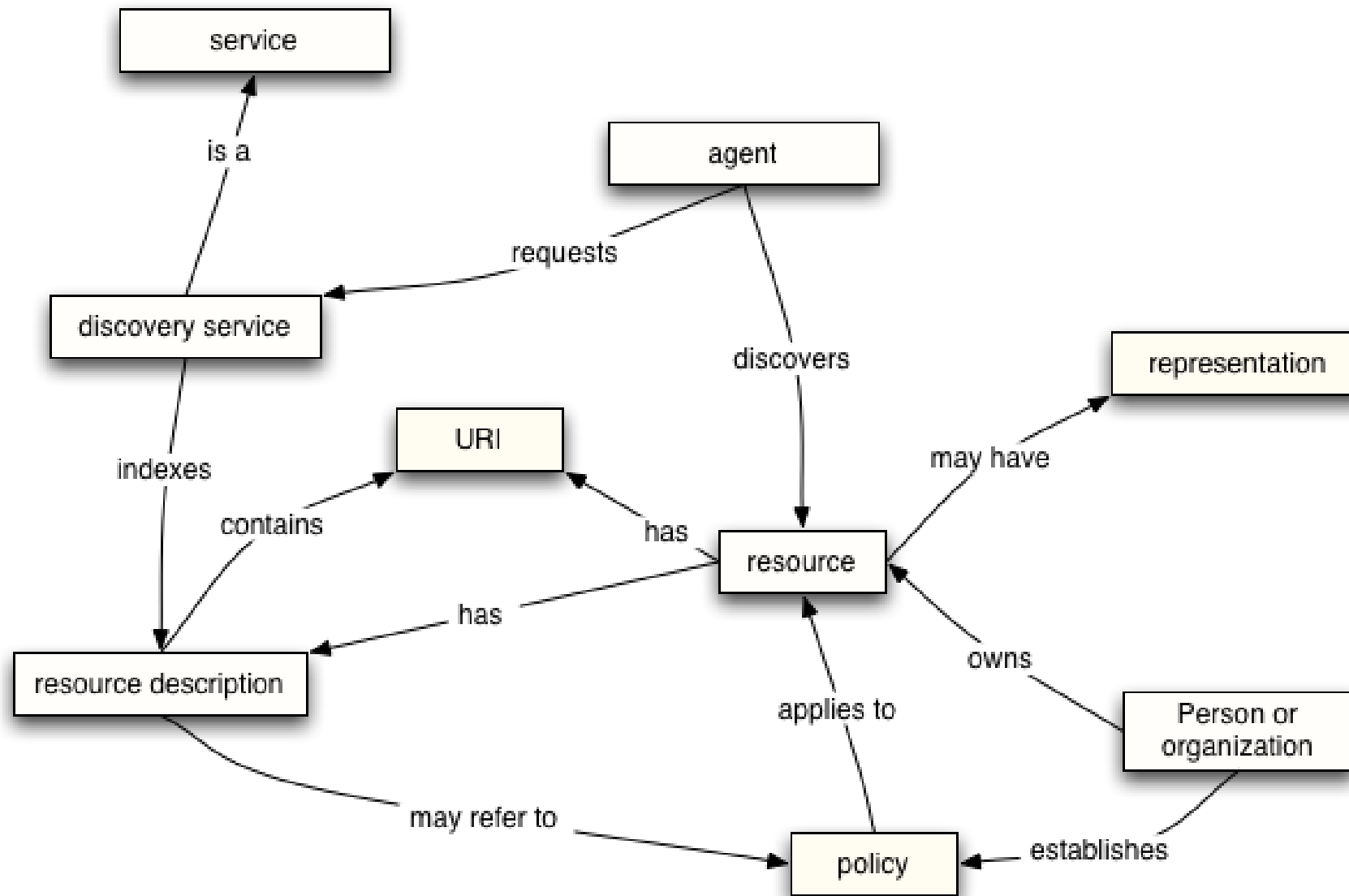
- Il modello orientato alla risorsa si concentra sulle risorse esistenti e proprietarie.



Modello orientato alla risorsa (ROM) – Concetti Base

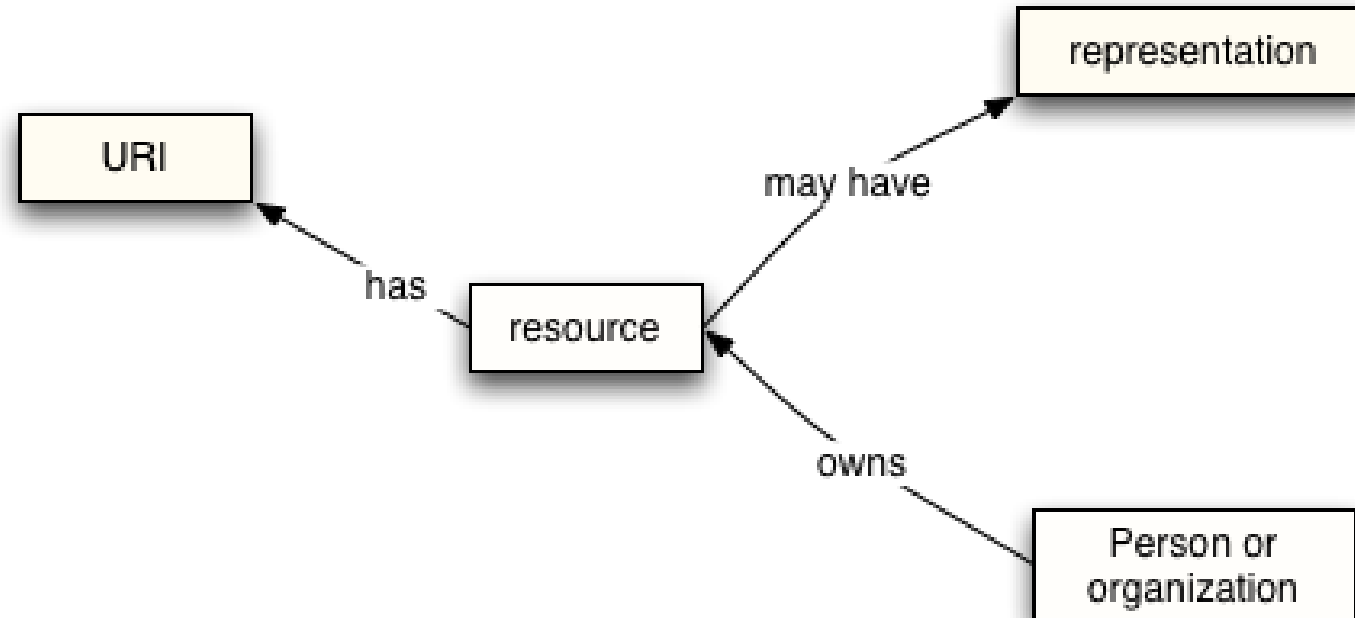
1. Discovery
2. Discovery Service
3. Identifier
4. Representation
5. Resource
6. Resource description

Modello orientato alla risorsa (ROM) – Dettaglio



Modello della Policy (PM)

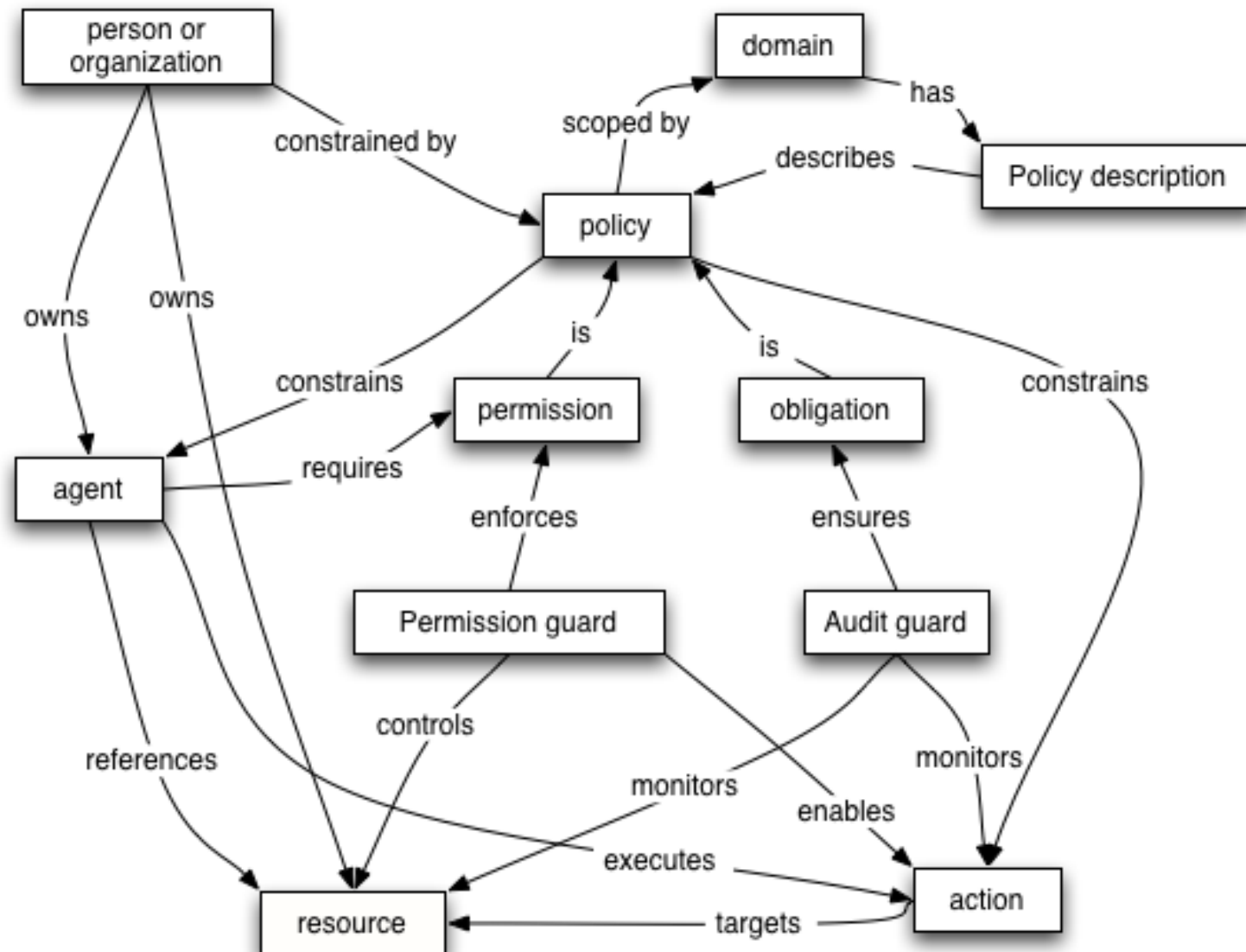
- Il modello della Policy si concentra sui vincoli e sul comportamento di agenti e servizi. La W3C estende questo modello a un ambito più generico inerente alle risorse poiché le policy possono essere applicate anche ai documenti (come le descrizioni dei servizi) e alle risorse computazionali attive.



Modello della Policy (PM) – Concetti Base

- Audit Guard
- Domain
- Obligation
- Permission
- Permission Guard
- Person Of Organization
- Policy
- Policy Description
- Policy Guard

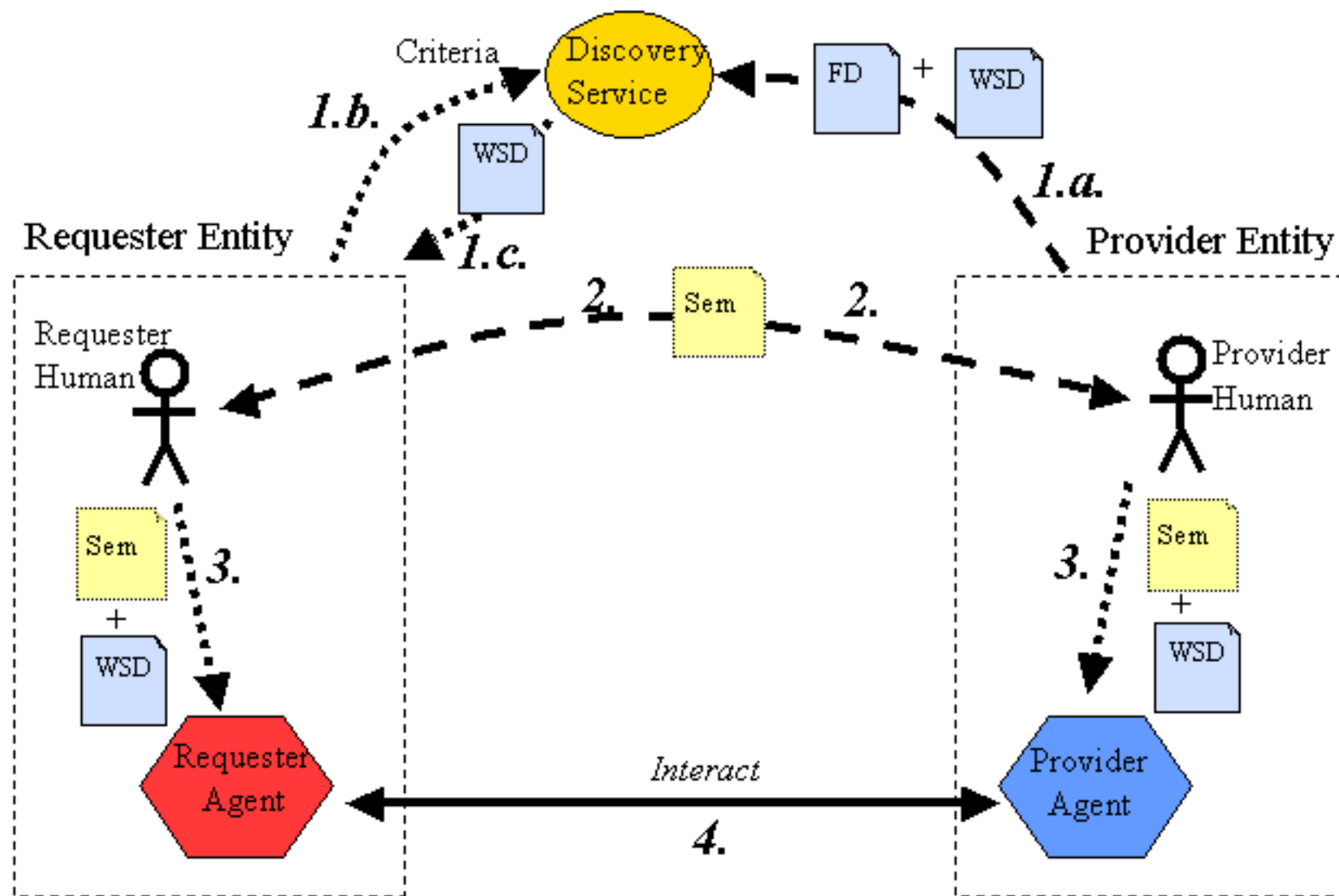
Modello della Policy (PM) – Dettaglio



Web Service Discovery

- Consente l'accesso ai sistemi software tramite Internet utilizzando i protocolli standard. Nello scenario di base c'è un fornitore di servizi Web che pubblica un servizio e un consumatore di servizi Web che utilizza questo servizio. Un Web Service Discovery è il processo di ricerca di servizi Web adatti per un determinato compito
- Universal Description, Discovery and Integration (UDDI) è un registro basato su XML per i servizi Internet aziendali. Un provider può registrare esplicitamente un servizio con un Registro di servizi Web come UDDI o pubblicare documenti aggiuntivi destinati a facilitare la scoperta come i documenti WSIL (Web Services Inspection Language). Gli utenti o i consumatori del servizio possono cercare i servizi Web manualmente o automaticamente. L'implementazione dei server UDDI e dei motori WSIL dovrebbe fornire semplici API di ricerca o GUI basata sul Web per aiutare a trovare i servizi Web.
- In generale, se l'entità richiedente desidera avviare un'interazione con un'entità fornitore e non sa già quale agente provider desidera coinvolgere, allora l'entità richiedente può aver bisogno di "scoprire" un candidato adatto. Discovery è "l'atto di individuare una descrizione elaborabile da un computer di un servizio Web che potrebbe essere stata precedentemente sconosciuta e che soddisfa determinati criteri funzionali." L'obiettivo è trovare un servizio Web appropriato.

Web Service Discovery



Web Service Semantics

- Affinché i software interagiscano l'uno con l'altro è necessario stabilire una serie di condizioni:
 - Deve esserci una connessione fisica tra loro, in modo tale che i dati di un processo possano raggiungere un altro
 - Ci deve essere un accordo (agreement) sulla forma dei dati, ad esempio se i dati sono linee di testo, strutture XML, ecc.
 - I due (o più) programmi devono condividere un accordo in merito al significato previsto dei dati. Ad esempio, se i dati sono destinati a rappresentare una pagina HTML da sottoporre a rendering o se i dati rappresentano lo stato corrente di un conto bancario; le aspettative e l'elaborazione coinvolte nell'elaborazione dei dati sono diverse, anche se la forma dei dati è identica.
 - Ci deve essere un accordo sull'elaborazione implicita dei messaggi scambiati tra i programmi. Ad esempio, è previsto che il servizio Web per gli ordini di acquisto - da parte dell'agente che ha effettuato l'ordine - elabori il documento contenente l'ordine di acquisto come ordine di acquisto, anziché semplicemente registrarlo a fini di controllo.
- La misura in cui l'accordo condiviso sulla forma, la struttura e il significato di un messaggio è condiviso oltre i soli agenti coinvolti nel messaggio governa la visibilità complessiva della semantica dei messaggi.

Web Service Semantics nei modelli architetturali

- I diversi modelli dell'architettura si concentrano su diversi aspetti delle problematiche inerenti l'interoperabilità tra agenti dei servizi Web.
 - Il Modello orientato ai messaggi si focalizza su come gli agenti dei servizi Web (agenti del richiedente e del fornitore) possono interagire tra loro utilizzando un modello di comunicazione orientato al messaggio. Il formato dei messaggi come infoset XML e la strutturazione dei messaggi in termini di buste, intestazioni e corpi, come descritto in tale modello, agisce come gettare le basi per la comprensione standard dei messaggi scambiati tra agenti dei servizi Web.
 - Il Modello orientato ai servizi si basa sulle basi della comunicazione dei messaggi aggiungendo il concetto di azione e servizio . In sostanza, il modello di servizio ci consente di interpretare i messaggi come richieste di azioni e come risposte a tali richieste. Inoltre, consente un'interpretazione dei diversi aspetti dei messaggi da esprimere in termini di aspettative diverse, in modi ben compresi, delle diverse parti del messaggio: in effetti, un approccio incrementale e stratificato al servizio è possibile utilizzando intestazioni ben comprese .
 - Il modello orientato alle risorse estende ulteriormente aggiungendo il concetto di risorsa . Le risorse sono importanti internamente all'architettura (un servizio Web è meglio inteso come una risorsa nel contesto della gestione dei servizi Web e in termini di gestione delle policy) ed esternamente: le risorse sono una metafora importante per interpretare l'interazione tra un'entità richiedente e un fornitore entità .

Web Service Semantics e i metadati

- Una parte importante dell'approccio di Service Oriented Architecture è l'uso esteso dei metadati.
- Ciò è importante per diversi motivi: promuove l'interoperabilità richiedendo maggiore precisione nella documentazione dei servizi Web e consente di fornire strumenti per garantire un più alto grado di automazione allo sviluppo di servizi Web (e quindi riduce il costo di distribuzione dello stesso).
- I metadati associati a un servizio Web possono essere considerati una descrizione parziale machine-readable (macchina-leggibile) della semantica del servizio Web.
- In particolare, utilizzando tecnologie come WSDL, un servizio Web può essere descritto in un documento leggibile dalla macchina sulle forme dei messaggi previsti, i tipi di dati degli elementi dei messaggi e utilizzando un linguaggio di descrizione della coreografia i flussi previsti di messaggi tra agenti dei servizi Web.

Web Services Security

- Le minacce ai servizi Web comportano minacce al sistema host, all'applicazione e all'intera infrastruttura di rete. Per proteggere i servizi Web, sono necessari una serie di meccanismi di sicurezza basati su XML per risolvere i problemi relativi all'autenticazione, al controllo degli accessi basato sui ruoli, all'applicazione dei criteri di sicurezza distribuiti, alla sicurezza dei livelli di messaggio che consente la presenza di intermediari.
- Al momento non ci sono specifiche generali per la sicurezza dei servizi Web. Di conseguenza, gli sviluppatori possono creare servizi che non utilizzano queste funzionalità o possono sviluppare soluzioni ad hoc che potrebbero portare a problemi di interoperabilità.
- Le implementazioni dei servizi Web possono richiedere meccanismi di sicurezza point-to-point e / o end-to-end, a seconda del grado di minaccia o rischio. I tradizionali meccanismi di sicurezza point-to-point orientati alla connessione potrebbero non soddisfare i requisiti di sicurezza end-to-end dei servizi Web. Tuttavia, la sicurezza è un equilibrio tra rischio valutato e costo delle contromisure. A seconda della tolleranza al rischio degli implementatori, la sicurezza del livello di trasporto point-to-point può fornire sufficienti contromisure di sicurezza.

Web Services Security

- Esistono molte sfide alla sicurezza per l'adozione di servizi Web. Al livello più alto, l'obiettivo è creare un ambiente in cui transazioni a livello di messaggio e processi aziendali possano essere condotti in modo sicuro in modo end-to-end. È necessario garantire che i messaggi siano protetti durante il transito, con o senza la presenza di intermediari. Potrebbe anche essere necessario garantire la sicurezza dei dati archiviati.
- I requisiti per fornire sicurezza end-to-end per i servizi Web possono così essere riepilogati:
 - Authentication Mechanisms (Meccanismi di autenticazione)
 - Authorization (Autorizzazione)
 - Data Integrity and Data Confidentiality (Integrità dei dati e riservatezza dei dati)
 - Integrity of Transactions and Communications (Integrità delle transazioni e delle comunicazioni)
 - Non-Repudiation
 - End-to-End Integrity and Confidentiality of Messages (Integrità end-to-end e riservatezza dei messaggi)
 - Audit Trails (Tracce di Controllo)
 - Distributed Enforcement of Security Policies (Applicazione distribuita delle politiche di sicurezza)

Web Services Security

- Le organizzazioni che implementano servizi Web devono essere in grado di condurre gli affari in modo sicuro. Ciò implica che tutti gli aspetti dei servizi Web, inclusi il routing, la gestione, la pubblicazione e l'individuazione, devono essere eseguiti in modo sicuro. Gli implementatori di servizi Web devono essere in grado di utilizzare servizi di sicurezza come autenticazione, autorizzazione, crittografia e auditing.
- I messaggi dei servizi Web possono scorrere attraverso i firewall e possono essere sottoposti a tunnel attraverso porte e protocolli esistenti. La sicurezza dei servizi Web richiede l'uso di politiche aziendali appropriate che potrebbero dover essere integrate con criteri di cross-enterprise e risoluzione di trust esterni. Le organizzazioni potrebbero dover implementare le funzionalità elencate di seguito:
 - Cross-Domain Identities
 - Distributed Policies
 - Trust Policies
 - Secure Discovery Mechanism (Meccanismo di rilevamento sicuro)
 - Trust and Discovery
 - Secure Messaging

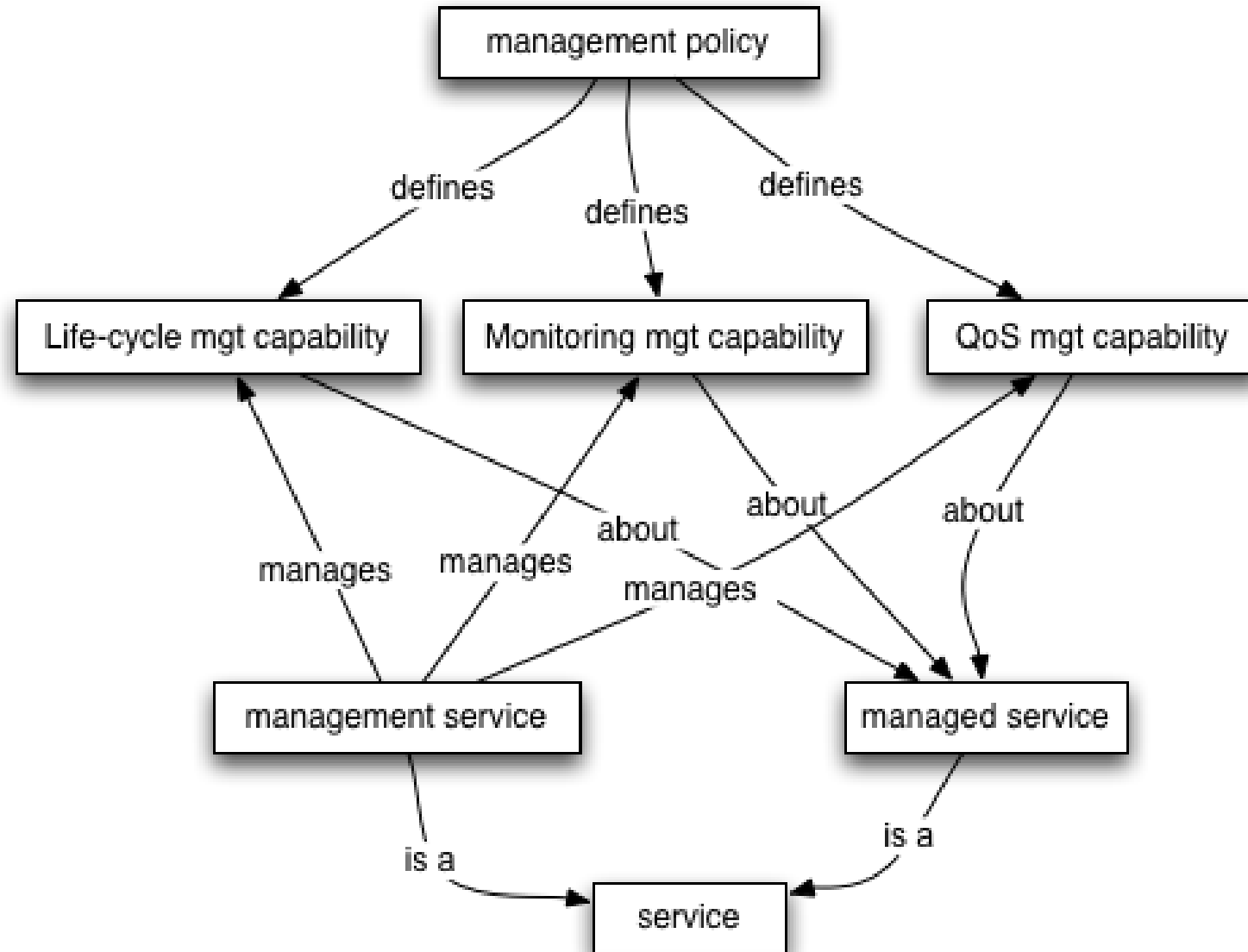
Affidabilità dei servizi Web

- I Servizi WEB devono garantire oltre la sicurezza anche un alto grado di affidabilità ovvero di robustezza; in particolare sono rilevanti i seguenti aspetti:
 - Message reliability (Affidabilità dei messaggi)
 - Service reliability (Affidabilità del servizio)
 - Reliability and management (Affidabilità e Gestione in senso generale)

Gestione dei servizi Web

- La gestione dei servizi Web deve avvenire attraverso una serie di attività che consentano il Monitoraggio, il Controllo e i Reports sulla qualità e l'utilizzo dei servizi, come ad esempio, disponibilità (presenza e numero di istanze di servizio) e prestazioni (ad esempio, latenza di accesso e tassi di errore) e anche accessibilità (degli endpoint). Le informazioni di interesse sull'utilizzo dei servizi includono frequenza, durata, ambito, estensione funzionale e autorizzazione di accesso.
- Un servizio Web diventa gestibile quando espone un set di management operations.
- Sebbene la fornitura di funzionalità di amministrazione consenta a un servizio Web di diventare gestibile, l'estensione e il grado di gestione consentiti sono definiti nelle policy associate al servizio Web. Le politiche di gestione vengono quindi utilizzate per definire gli obblighi e le autorizzazioni sul servizio Web.

Gestione dei servizi Web – Management Concepts and Relationships



WEB SERVICES

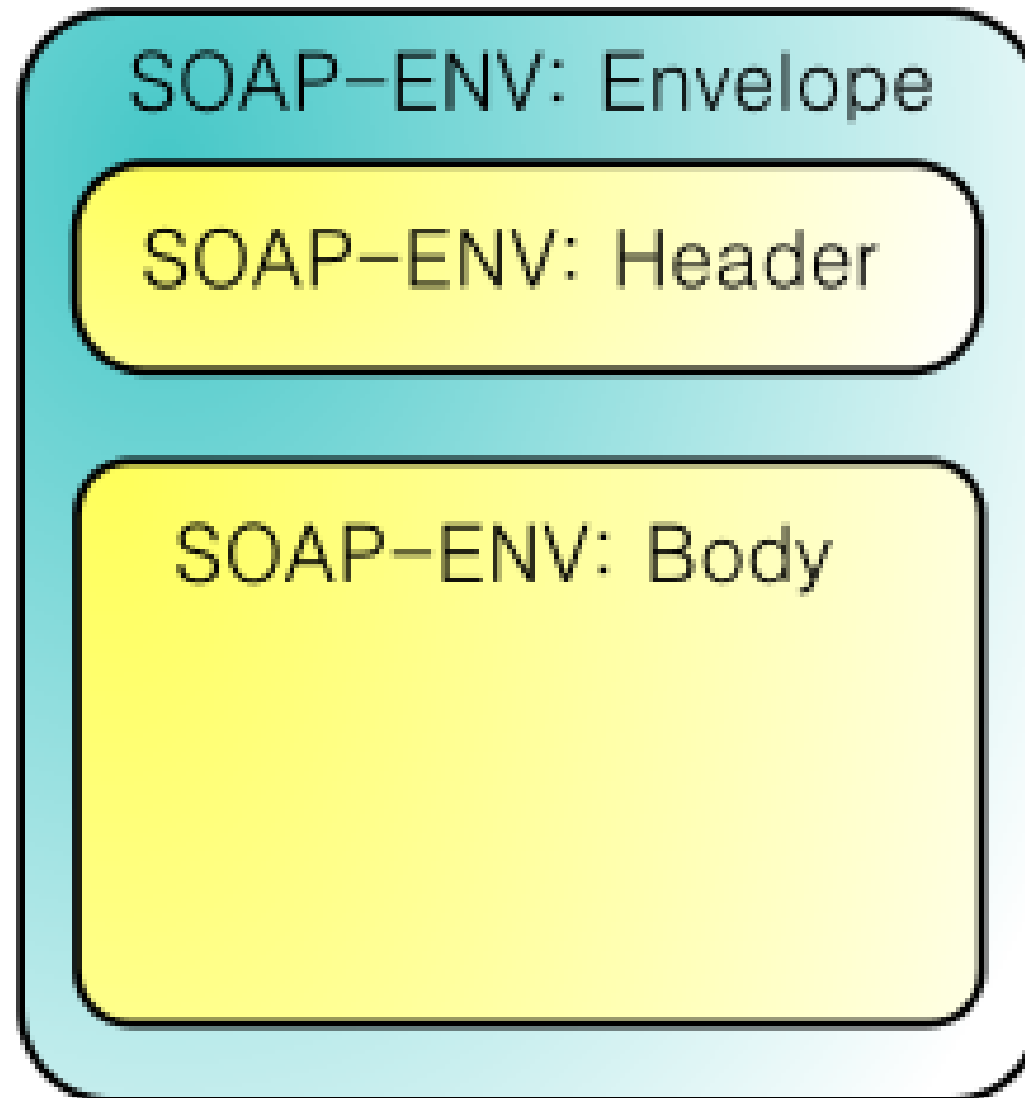
IL PROTOCOLLO SOAP

Il protocollo SOAP

- Il Simple Object Access Protocol (SOAP) è un protocollo basato su XML che consente a due applicazioni di comunicare tra loro sul Web.
- Pubblicato come Working Draft dal W3C nel dicembre 2001 (www.w3.org/TR/soap12-part0 , www.w3.org/TR/soap12-part1 , www.w3.org/TR/soap12-part2), SOAP definisce il formato dei messaggi che due applicazioni possono scambiarsi utilizzando i protocolli Internet, come ad esempio HTTP, per fornire dati e richiedere elaborazioni.
- Il protocollo è indipendente dalla piattaforma hardware e software ed è indipendente dal linguaggio di programmazione utilizzato per sviluppare le applicazioni comunicanti.
- Grazie a questo protocollo, il concetto di Web come piattaforma per la pubblicazione di documenti contenenti informazioni fruibili da esseri umani cambia radicalmente.
- È possibile affiancare alla pubblicazione classica delle pagine Web una pubblicazione delle informazioni destinate alle applicazioni.
- Questo nuovo tipo di pubblicazione costituisce i cosiddetti servizi Web. Un servizio Web è un'applicazione che fornisce funzionalità tramite il Web in maniera indipendente dalla piattaforma utilizzata. I meccanismi che consentono questa indipendenza sono XML e HTTP.

Messaggi SOAP

- Abbiamo detto SOAP è basato su XML. In effetti, un messaggio SOAP non è altro che un documento XML che descrive una richiesta di elaborazione o il risultato di una elaborazione. In particolare, un messaggio SOAP è costituito dai seguenti elementi:
 - Envelope: Rappresenta il contenitore del messaggio e costituisce l'elemento root del documento XML
 - Header: È un elemento opzionale che contiene informazioni globali sul messaggio; ad esempio, nell'header potrebbe essere specificata la lingua di riferimento del messaggio, la data dell'invio, ecc.
 - Body: Rappresenta la richiesta di elaborazione o la risposta derivata da una elaborazione
 - Fault: Se presente, fornisce informazioni sugli errori che si sono verificati durante l'elaborazione; questo elemento può essere presente soltanto nei messaggi di risposta
- È opportuno evidenziare che SOAP definisce soltanto la struttura dei messaggi non il loro contenuto.
- I tag per descrivere una richiesta di elaborazione o un risultato vengono definiti in uno schema specifico ed utilizzati all'interno della struttura SOAP sfruttando il meccanismo dei namespace.



SOAP: il framework

- Riepilogando è un protocollo per lo scambio di messaggi tra componenti software, tipicamente nella forma di componentistica software.
- La parola object manifesta che l'uso del protocollo dovrebbe effettuarsi secondo il paradigma della programmazione orientata agli oggetti.
- SOAP è la struttura operativa (framework) estensibile e decentralizzata che può operare sopra varie pile protocollari per reti di computer fornendo tramite messaggi richieste di procedure remote.
- I richiami di procedure remote possono essere infatti modellati come interazione di parecchi messaggi SOAP.
- SOAP dunque è uno dei protocolli che abilitano i servizi Web.
- SOAP può operare su differenti protocolli di rete, ma HTTP è il più comunemente utilizzato e l'unico ad essere stato standardizzato dal W3C, su cui è incapsulato (embedded) il relativo messaggio.
- SOAP si basa sul metalinguaggio XML e la sua struttura segue la configurazione head-body, analogamente ad HTML.

SOAP: il framework

- Il segmento opzionale "Header" contiene metadati come quelli che riguardano il routing, la sicurezza, le transazioni e parametri per l'orchestration.
- Il segmento obbligatorio "Body" trasporta il contenuto informativo e talora viene detto carico utile, o payload. Questo deve seguire uno schema definito dal linguaggio XML Schema. SOAP può essere utilizzato in due modi diversi per una chiamata:
 - Richiesta via SOAP di parametri: il client controlla nel Service Registry l'oggetto d'interesse e sviluppa il messaggio secondo i parametri contenuti nel Service Registry.
 - General Purpose Messaging: un programmatore può sviluppare un suo protocollo privato, il client conosce a priori i parametri e non necessita di consultare il service registry.
- All'interno del body del messaggio inserisco i dati scritti nel formato concordato con lo sviluppatore.

SOAP: Esempio di richiesta

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">  
  <soap:Body>  
    <getProductDetails xmlns="http://magazzino.example.com/ws">  
      <productId>827635</productId>  
    </getProductDetails>  
  </soap:Body>  
</soap:Envelope>
```

SOAP: Esempio di risposta

```
<soap:Envelope xmlns:soap="http://schemas.xmlsoap.org/soap/envelope/">
  <soap:Body>
    <getProductDetailsResponse xmlns="http://magazzino.example.com/ws">
      <getProductDetailsResult>
        <productName>Toptimate, set da 3 pezzi</productName>
        <productId>827635</productId>
        <description>Set di valigie; 3 pezzi; poliestere; nero.</description>
        <price>96.50</price>
        <inStock>true</inStock>
      </getProductDetailsResult>
    </getProductDetailsResponse>
  </soap:Body>
</soap:Envelope>
```

SOAP & Remote Procedure Call (RPC)

- Una parte molto complessa dell'elaborazione distribuita consiste nella gestione delle chiamate remote.
- Si tratta di chiamare del codice residente su un altro server, passandogli eventualmente dei parametri, e riceverne la risposta, anch'essa con un'opportuna codifica.
- Attualmente, esistono molti standard per la codifica e la trasmissione delle chiamate e delle risposte.
- Invece di usare complicati bridge per tradurre un protocollo in un altro, quando due framework diversi devono comunicare tra loro, SOAP si propone come protocollo universale per la trasmissione dei dati di RPC.
- Tuttavia, SOAP è stato studiato per avere usi che si estendono oltre la semplice RPC.
- SOAP, non si basa su tecnologie proprietarie e la sua applicazione è completamente libera.

Cos'è SOAP

- SOAP è un protocollo leggero che permette di scambiare informazioni in ambiente distribuito:
 - SOAP è basato su XML.
 - SOAP gestisce informazione strutturata.
 - SOAP gestisce informazione tipata.
 - SOAP non definisce alcuna semantica per applicazioni o scambio messaggi, ma fornisce un mezzo per definirla.

SOAP e l'XML

- Tutti i protocolli Citati sono binari, mentre SOAP è basato su XML, quindi testuale.
- Il debugging è notevolmente semplificato, perché l'XML è leggibile anche da essere umani.
- I dati sono molto piu firewall-friendly: un firewall puo analizzarli e dedurre che sono innocui, facendoli passare.
- Come detto SOAP è stato pensato per usare HTTP come trasporto.
- Il principale svantaggio di SOAP è costituito proprio dalla sua natura testuale che lo rende molto meno performante rispetto alle sue controparti binarie (CORBA e .NET Remoting in particolare).

SOAP Gestisce Informazione Strutturata e Tipata

- La strutturazione dei messaggi SOAP, che deriva direttamente dalla strutturazione implicita di XML, è molto adatta al trasporto dei dati.
- La gestione dei tipi utilizza il type system standard degli Schemi XML, molto robusto e versatile.
- In questo modo è possibile far viaggiare insieme ai dati anche dei metadati che ne descrivono la struttura.

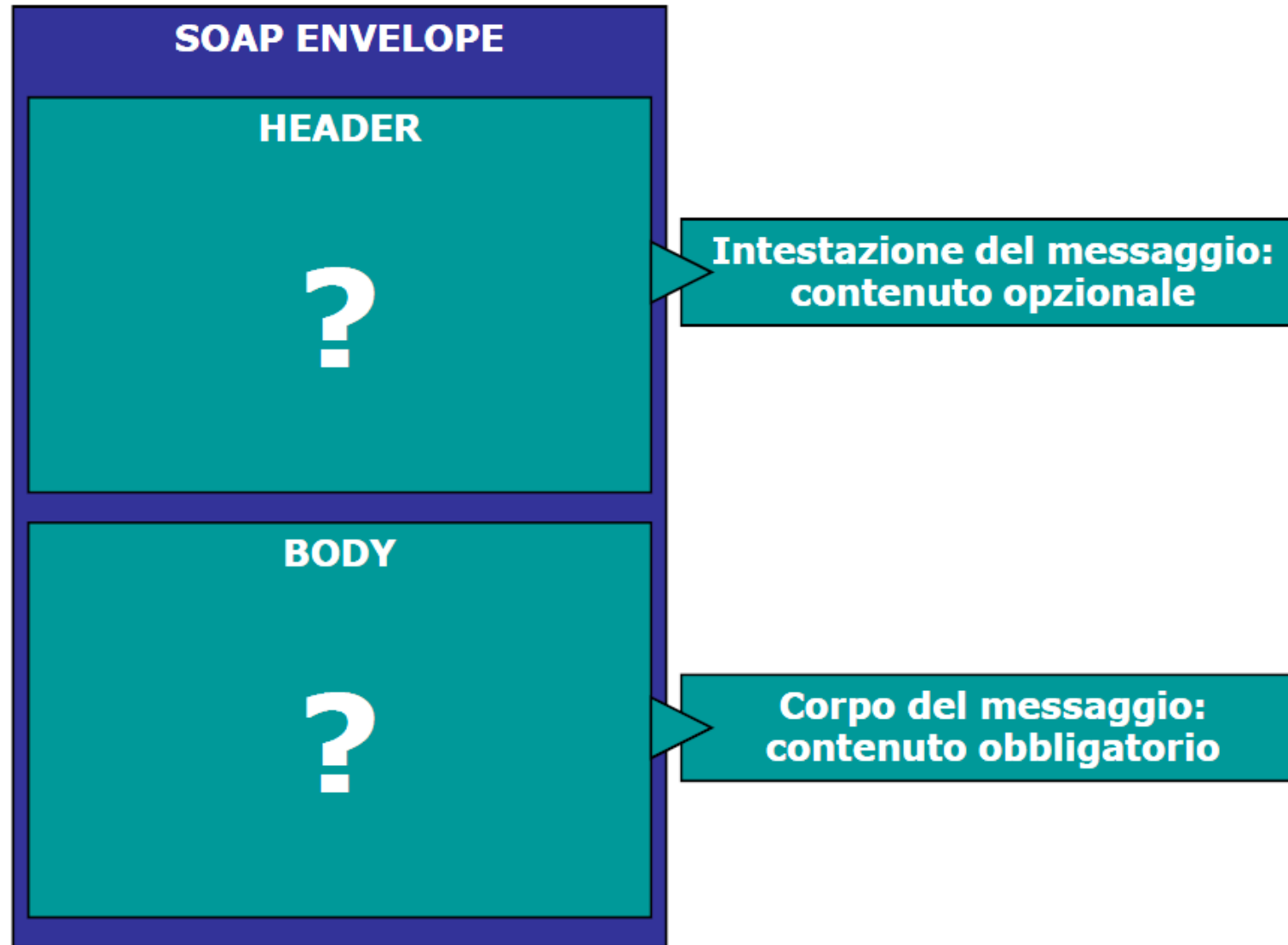
SOAP non ha una Semantica Predefinita

- SOAP non definisce semantiche per i dati e le chiamate, ma fornisce agli sviluppatori i mezzi per farlo.
- Con un intenso uso dei Namespaces XML (un messaggio SOAP è composto da elementi provenienti da almeno tre namespaces diversi)
- SOAP permette agli autori dei messaggi di dichiararne la semantica usando grammatiche XML definite per lo scopo in particolari namespaces.

Applicazioni SOAP

- Parlare di SOAP solo in ambito RPC è in realta errato.
- SOAP in effetti e un generico framework per lo scambio di informazioni.
- Nella specifica di SOAP redatta dal W3C è pero inserita una sezione in cui viene descritto un meccanismo standard per la codifica delle informazioni RPC con SOAP.

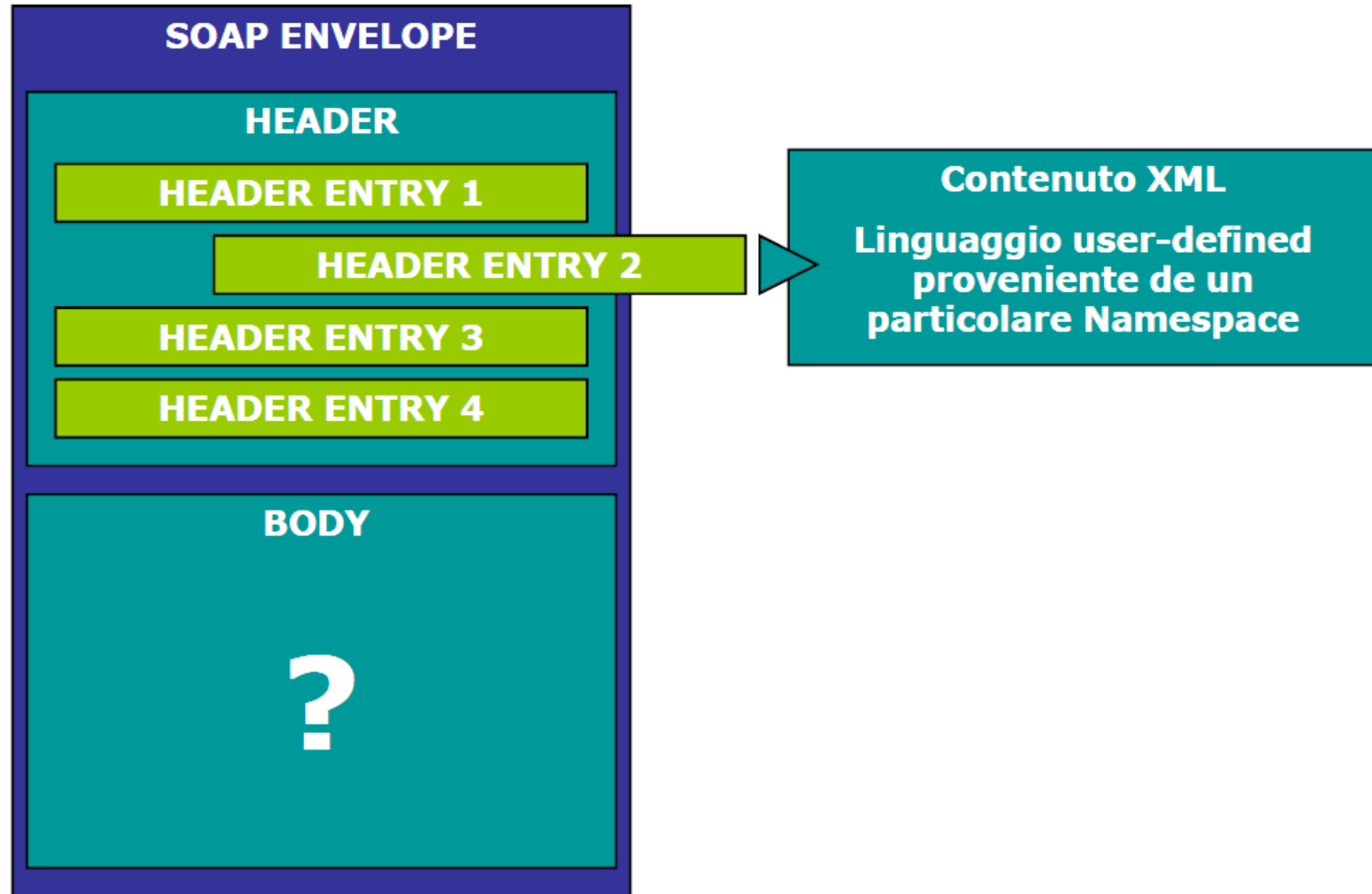
Contenuti Obbligatori Messaggio SOAP



Headers SOAP

- Gli headers forniscono un modo per estendere un messaggio in maniera modulare e senza la necessita di rinegoziare l'intero formato tra tutte le parti in causa.
- L'elemento header deve essere il primo figlio di envelope.
- Tutti gli elementi figli di header vanno considerati singole header entries.
- Come per il Body, non esiste un formato per le header entries: la grammatica va importata da altri namespaces.

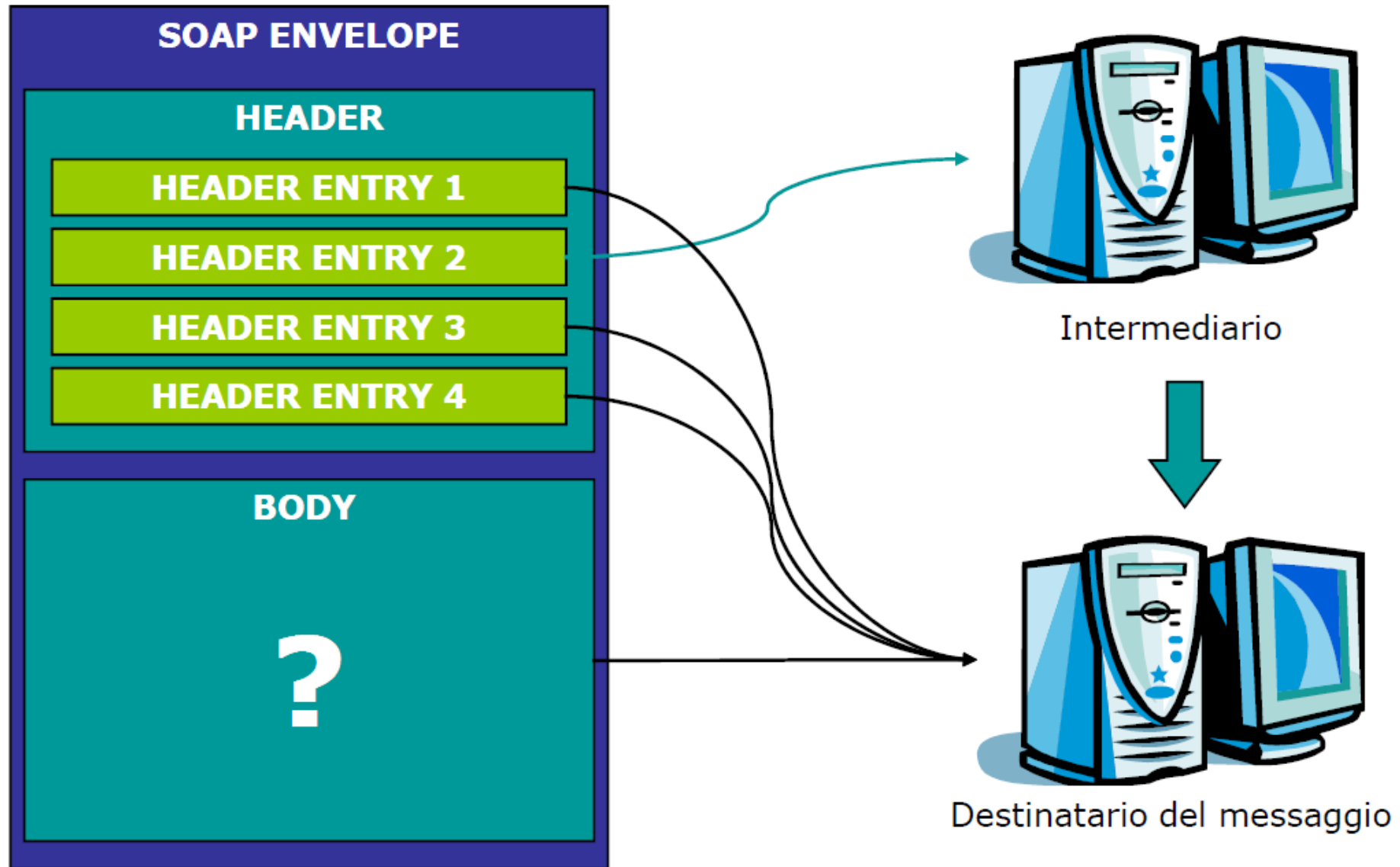
Headers SOAP



Actors SOAP

- Un messaggio SOAP può attraversare varie applicazioni prima di raggiungere la sua destinazione.
- Le header entry, oltre che al destinatario finale, possono essere destinate a queste applicazioni “intermedie”.
- L’attributo globale actor può essere applicato a ogni header entry per specificare, con una URI identificativa, l’applicazione destinataria della entry.
- Se non viene specificato, l’actor predefinito è sempre il destinatario del messaggio.

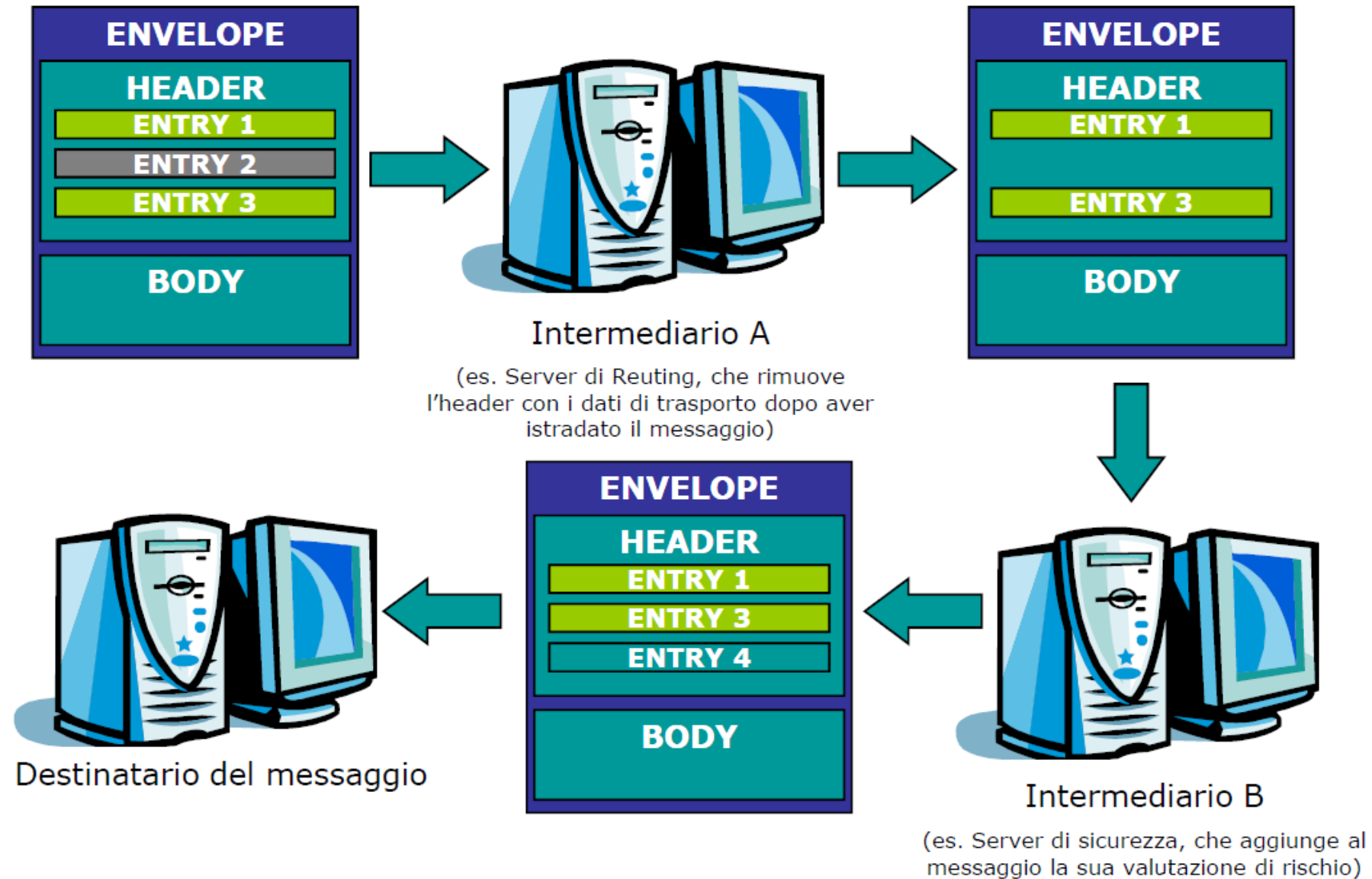
Actors SOAP



Headers con Actors SOAP

- L'attore destinatario di una header entry deve sempre rimuoverla prima di inoltrare il messaggio all'attore successivo lungo il path che conduce al destinatario.
- Un attore può, se vuole, inserire altre header entries nel messaggio, ma non toccarne il corpo.
- La URI speciale <http://schemas.xmlsoap.org/soap/actor/next> indica che un header dovrà essere elaborato dal primo (o prossimo) attore che incontrerà sul path.

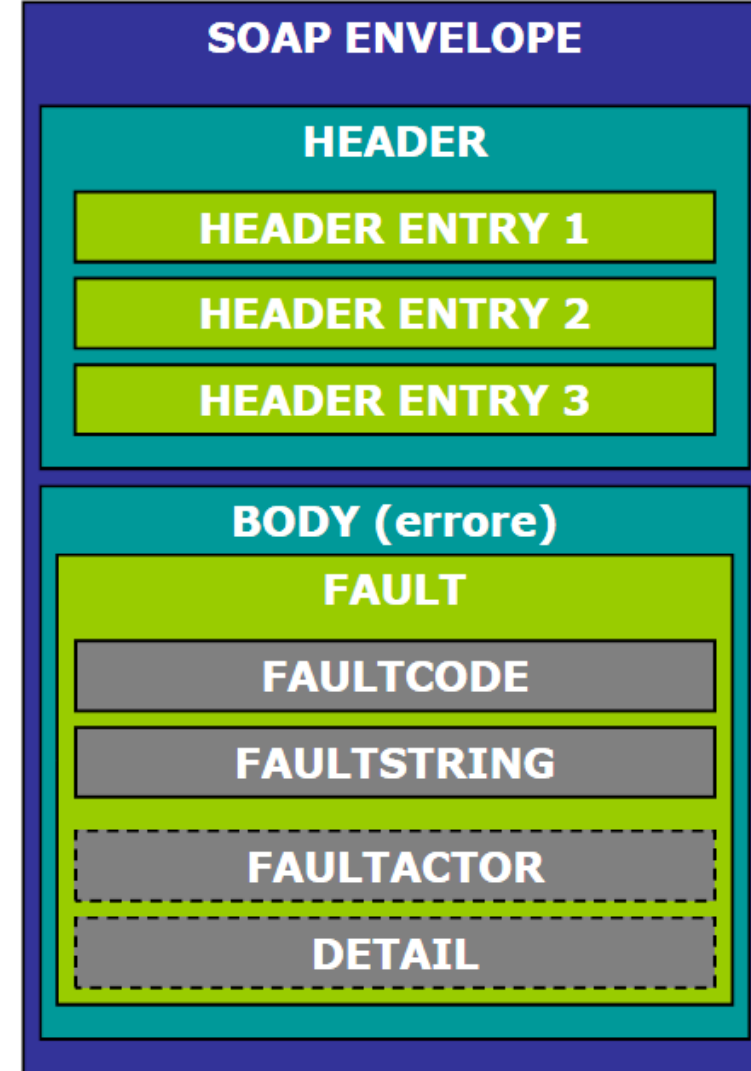
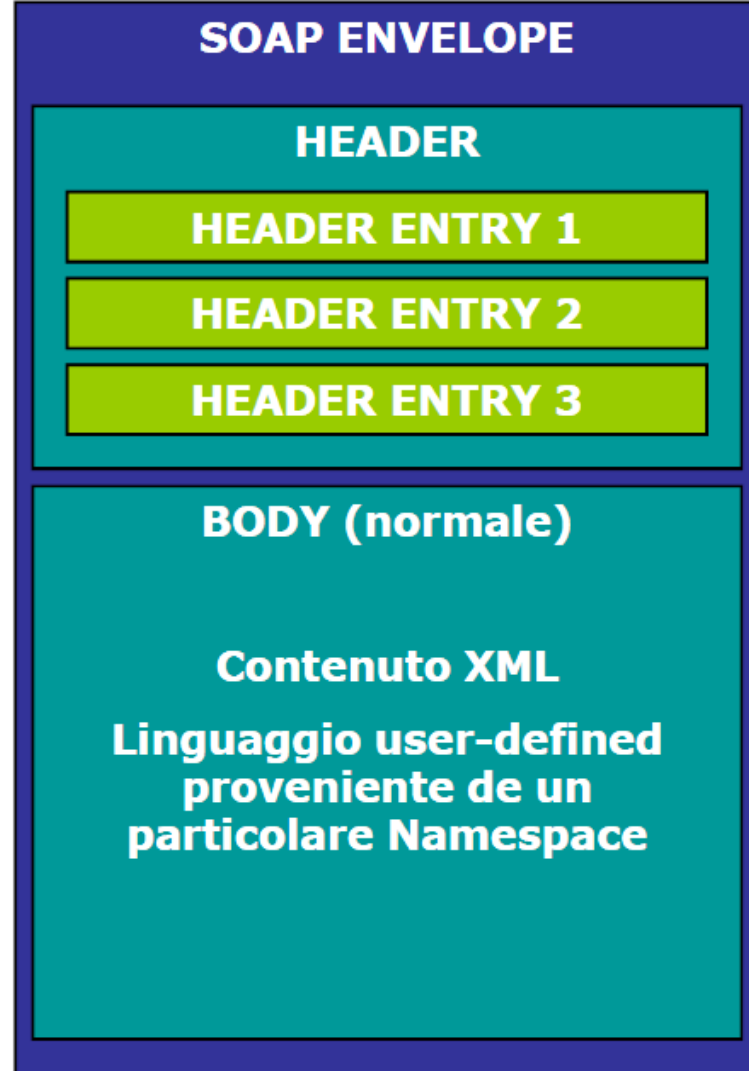
Headers con Actors SOAP



Body SOAP

- Il corpo SOAP contiene le informazioni per il destinatario finale del messaggio.
- Il contenuto di Body è definito dall'implementatore del messaggio tramite l'uso di un particolare namespace.
- Il destinatario dovrà elaborare i dati e obbedire alla loro semantica, specificata dal namespace di appartenenza.
- L'elemento Body è semanticamente equivalente a una header entry con `mustUnderstand="1"` e actor non specificato.

Body SOAP



Errori SOAP

- Esiste un solo elemento predefinito in SOAP che può apparire nel Body del messaggio: l'elemento Fault.
- Fault compare nei messaggi SOAP di risposta se il destinatario o un intermediario non sono stati in grado di elaborare la loro parte del messaggio di richiesta.
- Se presente, l'elemento Fault deve essere uno dei figli di Body e può comparire solo una volta.
- L'elemento Fault serve a fornire informazioni su errori derivanti dall'elaborazione del messaggio.
- Il contenuto dell'elemento Fault è:
 - Un elemento faultcode, obbligatorio. Fornisce un codice di identificazione per l'errore, ad uso del software. SOAP definisce alcuni codici standard per questo elemento.
 - Un elemento faultstring, obbligatorio. Fornisce una descrizione testuale dell'errore.
 - Un elemento faultactor, obbligatorio. Specifica l'attore che ha generato l'errore, se non si tratta del destinatario (cioè se l'errore è stato provocato dalla mancata elaborazione di una header entry).

Errori SOAP

- Se l'errore è provocato dall'elaborazione del Body, l'elemento Fault deve fornire anche i dettagli dell'errore in un elemento detail.
- Tuttavia, l'elemento detail non deve apparire se l'errore è stato provocato dall'elaborazione di una header entry: in questo caso i dettagli vanno inseriti in una nuova header entry.
- Il contenuto dell'elemento detail deve essere specificato usando una grammatica XML importata da un namespace opportuno.

Faultcode Predefiniti

- SOAP definisce quattro codici generici di errore, contenuti nel suo namespace.
 - VersionMismatch: errore dipendente dalla versione del messaggio.
 - MustUnderstand: errore nell'elaborazione di una header entry.
 - Client: errore dipendente dal mittente.
 - Server: errore dipendente del destinatario.
 - Ciascun codice può essere esteso con codici user-defined piu specifici: ad esempio, Client.Authentication.

SOAP Encoding

- La codifica dei tipi è utilizzata per la serializzazione dei valori nei messaggi.
- La codifica standard dei tipi di SOAP è indicata ponendo l'attributo `encodingStyle` di un elemento al namespace
 - <http://schemas.xmlsoap.org/soap/encoding/>
- Tuttavia, gli sviluppatori possono definire ed utilizzare anche altre codifiche dei tipi.
- Gli aspetti fondamentali della codifica SOAP per i tipi, che è usata anche in altri linguaggi come il WSDL non verranno qui discussi ma lasciati come materia di studio; di seguito l'elenco di massima:
 - Codifica e Serializzazione dei Valori Semplici
 - Valori con Accessor Multipli
 - Strutture e Relativa Serializzazione (Una struttura dati in SOAP è codificata tramite un elemento che è l'accessor dell'intera struttura, contenente altri elementi nidificati, che sono gli accessor dei singoli membri della struttura. Una struttura può essere single- o multi-reference, cioè può avere un attributo `id` per essere "puntata" all'esterno da altri accessor. Gli accessor dei membri invece sono sempre locali, e non possono essere puntati dall'esterno.)
 - Array e Relativa Serializzazione

SOAP over HTTP

- SOAP è stato esplicitamente pensato per usare HTTP come protocollo “di trasporto”.
- Tuttavia, sono stati codificati anche i metodi per inserire messaggi SOAP in altri tipi di protocolli internet, ad esempio l'SMTP.
- Un messaggio SOAP viene trasportato in un messaggio HTTP.
- I framework SOAP come Apache SOAP rendono quasi del tutto trasparente questo approccio.

WEB SERVICES

REST & SOAP

Intrdouzione ai WS

- Il Web è nato negli anni '90 come una piattaforma per la condivisione di documenti distribuiti su diverse macchine e tra loro interconnessi.
- Il tutto è nato e si è evoluto grazie alla standardizzazione di alcuni semplici concetti:
 - URI: meccanismo per individuare risorse in una rete
 - HTTP: protocollo semplice e leggero per richiedere una risorsa ad una macchina
 - HTML: linguaggio per la rappresentazione dei contenuti
- Questa semplice idea iniziale si è evoluta nel corso degli anni non tanto nei concetti di base, quanto nel modo di intenderli e di utilizzarli.
- Al testo si sono aggiunti contenuti multimediali, i documenti sono generati dinamicamente e non pubblicati come pagine statiche, e poi il Web esce dalla visione esclusivamente ipertestuale per diventare un contenitore di applicazioni software interoperabili: una piattaforma applicativa distribuita.

Intrdouzione ai WS

- Questa prospettiva ha dato origine, intorno all'anno 2000, al concetto di Web Service: un sistema software progettato per supportare un'interazione tra applicazioni, utilizzando le tecnologie e gli standard Web.
- Il meccanismo dei Web Service consente di far interagire in maniera trasparente applicazioni sviluppate con linguaggi di programmazione diversi, che girano su sistemi operativi eterogenei.
- Questo meccanismo consente di realizzare porzioni di funzionalità in maniera indipendente e su piattaforme potenzialmente incompatibili facendo interagire i vari pezzi tramite tecnologie Web e creando un'architettura facilmente componibile.

SOAP e REST

- Allo stato attuale esistono due approcci alla creazione di Web Service:
 - Un approccio è basato sul protocollo standard SOAP (Simple Object Access Protocol), per lo scambio di messaggi per l'invocazione di servizi remoti, si prefigge di riprodurre in ambito Web un approccio a chiamate remote, Remote Procedure Call, tipico di protocolli di interoperabilità come CORBA, DCOM e RMI.
 - Un secondo approccio è ispirato ai principi architetturali tipici del Web e si concentra sulla descrizione di risorse, sul modo di individuarle nel Web e sul modo di trasferirle da una macchina all'altra. Questo approccio prende il nome di REST (REpresentational State Transfer).
- Vedremo in cosa consiste l'approccio REST o SOAP per la realizzazione di Web Service, vedremo come rappresentare risorse e come gestire il loro stato, come consumare ed implementare servizi RESTful e SOAP e quali strumenti sono disponibili per semplificare la realizzazione di questo tipo di servizi.